

ZLang HDL

Language Reference

Community Release

v0.1.0a24

Installation, language, simulation, RTL generation, and verification

Apache-2.0 Community distribution

Contents

1 Why ZLang HDL **3**
1.1 What makes ZLang different . . . 3
1.2 From intent to hardware . . . 3
1.3 Core principles . . . 3

2 Quick start: one complete design **4**
2.1 Source files and notation . . . 4

3 Implementation intent and hardware selection **5**
3.1 Behavior and implementation . . . 5
3.2 `implement` . . . 5
3.3 Implementation alternatives . . . 5
3.4 Hard constraints and objectives . . . 5
3.5 Latency, II, capacity, and current sharing boundary . . . 6
3.6 Frequency intent and physical timing . . . 7
3.7 Current HLS boundary . . . 8

4 Simulation, waveforms, and verification **8**
4.1 Native simulation . . . 8
4.2 Inputs, clocks, resets, and event schedules . . . 8
4.3 U/X diagnostics . . . 9
4.4 Waveforms and differential RTL comparison . . . 9
4.5 Debug native simulation in VS Code . . . 9
4.6 Source-level verification . . . 11
4.7 Evidence terminology . . . 12

5 Types and exact numerical semantics **12**
5.1 Characters, fixed strings, and tuples . . . 12
5.2 Nominal enums . . . 13
5.3 Nominal tagged unions . . . 13
5.4 Scalar families . . . 14
5.5 Integer width rules . . . 14
5.6 Fixed point . . . 15
5.7 Vectors and structs . . . 16
5.8 Slicing, concatenation, and representation . . . 17

6 Expressions, functions, generics, and compile-time hardware **19**
6.1 Operators and selection . . . 19
6.2 Pure functions . . . 20
6.3 Compile-time integer modulo . . . 21
6.4 Compile-time functions and selection . . . 21
6.5 Functional datapath . . . 22
6.6 Generic structs, functions, and operators . . . 23

7 State, clocks, rules, FSMs, and pipelines **24**
7.1 Clock and reset . . . 24
7.2 Registers and next state . . . 25
7.3 Delays and fixed pipelines . . . 27
7.4 Exact module timing contracts . . . 27
7.5 Guarded atomic actions . . . 28
7.6 Recommended asynchronous reset . . . 29

8 Hierarchy, interfaces, protocols, and CDC **30**
8.1 Modules and instances . . . 30
8.2 Clock and reset boundary . . . 32
8.3 Public top ABI . . . 33
8.4 Ready/valid . . . 33
8.5 Credit and request/response . . . 33
8.6 Aggregate protocols and the standard library . . . 34

8.7 Packets, virtual channels, and arbitration . . . 34
8.8 CSR interfaces . . . 34
8.9 Clock-domain crossings . . . 34
8.10 Exact conformance . . . 35

9 Storage and memories 36
9.1 FIFO . . . 36
9.2 Synchronous memory . . . 36
9.3 Byte masks . . . 37
9.4 Named same-clock ports . . . 37
9.5 Reset and initialization . . . 37
9.6 Explicit dual-clock memory . . . 37
9.7 ROM . . . 38
9.8 Mapping boundary . . . 38

10 Standard library and buses 38
10.1 Streams, storage, math, and coding . . . 38
10.2 Bus profiles . . . 38
10.3 CSR and RegBus . . . 39
10.4 AXI4-Lite, APB, AHB-Lite, and Wishbone . . . 39
10.5 AXI-Stream . . . 39
10.6 Bounded burst and AXI4 profiles . . . 39
10.7 Imports and current boundaries . . . 39

11 Projects, dependencies, and reproducibility 39
11.1 Project manifest . . . 40
11.2 Lock update . . . 40
11.3 Offline compilation . . . 41

12 Profiles, targets, and physical implementation 41
12.1 Compiler-shipped source descriptions . . . 42
12.2 Manual selection . . . 42
12.3 Boundaries . . . 43
12.4 User contract . . . 43
12.5 How target evidence affects selection . . . 43
12.6 Current boundary . . . 44
12.7 Clock constraints . . . 44

13 Generate and integrate RTL 44
13.1 Supported composition subset . . . 45
13.2 Explicitly unsupported . . . 47
13.3 Example and structural coverage . . . 47

14 Installation and external tools 47
14.1 Recommended installation with uv . . . 47
14.2 Conventional venv and pip alternative . . . 48
14.3 Windows through WSL2 . . . 48
14.4 Check and run ZLang . . . 48
14.5 Install the VS Code extension, LSP, and native debugger . . . 49
14.6 Choose which external tools you need . . . 49
14.7 Recommended: OSS CAD Suite . . . 50
14.8 Alternative: install components separately . . . 50
14.9 Verify the installation . . . 50
14.10 Troubleshoot formal runs . . . 51

15 Current limitations 51
15.1 Language and scheduling . . . 51
15.2 Simulation and verification . . . 51
15.3 CDC, storage, and physical targets . . . 51
15.4 Protocol and library boundaries . . . 51
15.5 Language and RTL boundaries . . . 52
15.6 Verification boundaries . . . 52

A Appendix A — Concise source reference **53**
A.1 Non-negotiable rules . . . 53
A.2 Smallest valid modules . . . 53
A.3 Types, literals, and exact conversions . . . 54
A.4 Aggregates and functions . . . 54
A.5 State, hierarchy, and protocols . . . 55
A.6 Functional datapath and implementation intent . . . 55
A.7 Verification . . . 56
A.8 Commands and completion check . . . 56

1 Why ZLang HDL

ZLang HDL is a typed hardware description language and compiler for designs whose numerical, state, timing, and protocol meaning must remain explicit. The production backend emits Direct SystemVerilog from checked semantics. Unsupported combinations are errors; they are not silently reinterpreted as different hardware.

1.1 What makes ZLang different

Concern	ZLang approach
Widths	Exact; no silent resize
Fixed point	Explicit quantization
State	Explicit next-state and atomic actions
Timing	Exact latency contracts
CDC	Explicit semantic crossing
Implementation choice	`implement` plus `intent`
Optimization	Exact legal alternatives only
Physical mapping	Target- and resource-aware
Verification	Evidence status is retained
RTL	Direct-SV from typed semantics

1.2 From intent to hardware

```
Designer intent
      ↓
Exact typed semantics
      ↓
Exact implementation alternatives
      ↓
Target/resource planning
      ↓
Verification and evidence
      ↓
Direct SystemVerilog
      ↓
FPGA / ASIC toolchain
```

Source defines required behavior. The compiler may vary structure only through exact alternatives admitted by the language and selected profile. Timing and state contracts remain part of the design. Resource estimates, synthesis measurements, routed measurements, bounded checks, and proofs retain distinct evidence labels.

1.3 Core principles

- Width and signedness changes are explicit with operations such as `extend`, `truncate`, and `bitcast`.
- Fixed-point rounding and overflow are explicit at `quantize` boundaries.
- Clock, reset, domain ownership, state updates, and protocol transfers have exact cycle meaning.
- Rules commit atomically from one pre-edge snapshot.
- Protocol adaptation and clock-domain crossing are explicit.
- Compilation and published artifacts are deterministic for the same inputs.

- Unsupported forms stop with a diagnostic.

2 Quick start: one complete design

The following design combines exact arithmetic, explicit state, an atomic rule, a safety property, and bounded implementation selection.

```
module RunningSum {
    clock clk reset rst
    in enable : bit
    in a, b : u8
    out reg total : u10 = 0
    out pair_sum : u9

    pair_sum = implement {
        a + b
        intent { latency == 0 ii == 1 minimize lut }
    }

    accumulate: when enable {
        total <- truncate<10>(total + extend<10>(a + b))
    }

    assert total_is_bounded @ clk { total <= 1023 }
}
```

`=` drives combinational hardware. Addition produces its exact result width, so two `u8` operands produce `u9`; a wider state update uses explicit `extend` and `truncate`. `<-` schedules next state on the active edge. The rule reads one pre-edge snapshot and commits atomically. `out reg` is architectural storage and holds its value when no update occurs.

The `implement` expression is the required value. Its `intent` constrains legal implementations without changing that value. Here the result is combinational (`latency == 0`) and accepts a new value every cycle (`ii == 1`).

Check, simulate, verify, and generate production RTL:

```
zlang design.zhl --top RunningSum --check
zlang sim design.zhl --top RunningSum --events events.jsonl --trace build/run.fst
zlang design.zhl --top RunningSum --verify --verification-work-dir build/verify
zlang design.zhl --top RunningSum --systemverilog build/RunningSum.sv
```

The generated Direct-SV is the production RTL artifact. Native simulation runs the checked design without generating RTL first; differential comparison can replay the same event stream against Icarus or Verilator. Verification reports preserve whether a result is bounded evidence or an unbounded proof.

2.1 Source files and notation

A source file may contain imports, type aliases, structs, pure functions, operator overloads, rewrite declarations, target/resource declarations, and multiple modules. Select a top explicitly with `--top` when needed. Names are case-sensitive; there are no semicolons.

Form	Meaning
`name = expression`	Drive a combinational value or resource control.
`state <- expression`	Schedule next state at the active edge.
`source -> destination`	Connect typed endpoints or bind a command.
`key => expression`	Select a value arm.
`thing @ clk`	Associate a declaration with a clock domain.
`object.field`	Select an aggregate, protocol, or resource member.

3 Implementation intent and hardware selection

3.1 Behavior and implementation

An expression defines required hardware meaning. Intent constrains and ranks legal ways to realize that meaning; it never changes the value being computed.

3.2 `implement`

Use `implement` when an expression defines one exact hardware value but more than one legal implementation may exist. The expression remains the required result. The nested `intent` block filters legal choices and optionally states which cost to prefer:

```
module IntentDotProduct {
    clock clk reset rst
    in a : vec<4,u3>
    in b : vec<4,u3>
    out y : u8

    y = implement {
        dot(a, b)
        intent {
            latency <= 3
            ii == 1
            dsp <= 4
            fmax >= 100
            minimize latency
        }
    }
}
```

3.3 Implementation alternatives

The compiler keeps the original exact expression as a legal choice and may also consider a bounded set of equivalent structures and physical implementations. It never relaxes a hard constraint. If no candidate satisfies every constraint, compilation fails and reports the rejected choices.

3.4 Hard constraints and objectives

An `intent` block is non-empty. Clauses are separated by whitespace or newlines; commas are not used. A metric may have at most one constraint, and the block may have at most one objective.

Form	Accepted values	Meaning
`lut REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the LUT estimate.
`ff REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the flip-flop estimate.
`dsp REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the DSP-resource estimate.
`bram REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the block-memory estimate.
`latency REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound accepted-input to corresponding-output latency according to the selected contract. A positive value requires a clock/reset context.
`ii REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 1`	Bound initiation interval: the nominal minimum spacing between accepted transactions when downstream capacity is available.
`fmax REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 1` MHz	Bound the available frequency estimate or compatible measurement. `fmax_est` is accepted as an alias.
`minimize METRIC`	`lut`, `ff`, `dsp`, `bram`, or `latency`	Rank legal choices by one cost.

Form	Accepted values	Meaning
`maximize fmax`	`fmax` or `fmax_est`	Prefer the highest compatible frequency evidence.

If an objective is omitted, selection defaults to `minimize lut`. Frequency may only be maximized; resources and latency may only be minimized. II has no objective form. Constraints filter the available architectures rather than creating new ones: for example, `ii <= 4` may select an existing II=1 choice.

`pipeline(N) { expression }` is different from `implement`. It always specifies exactly `N` cycles of visible latency. An `implement` block instead allows the compiler to select among the existing legal choices admitted by its constraints. Use `choice(...)` when the source explicitly supplies alternative expressions.

3.5 Latency, II, capacity, and current sharing boundary

For a ready/valid boundary, transactions occur only on transfers:

```
input transfer  = input.valid  && input.ready
output transfer = output.valid && output.ready
```

Latency is the unstalled distance from an accepted input to its corresponding valid output. II is the nominal minimum spacing between accepted inputs when the implementation and downstream path have capacity. Capacity is the number of accepted but not yet retired transactions the block can hold. Downstream backpressure may increase the observed wall-clock interval; it does not change the declared II.

Current support has three distinct cases:

Form	Current behavior
Ordinary scalar `implement`	Fixed-rate only. Available candidates have II=1; an II>1 constraint does not create resource sharing.
`transform pipeline(auto, ...)` over a supported ready/valid kernel	Globally stalled pipeline with unstalled II=1. Output data and `valid` hold while downstream `ready` is low.
`transform pipeline(auto) { implement { a*b + c*d ... } }`	Narrow non-interleaved shared arithmetic: one multiplier, latency 4, II 4, capacity 1.

The supported II=4 form requires one ready/valid input and output, integer operands, the exact `a*b + c*d` shape, one clock/reset domain, `dsp <= 1`, and an intent that admits II 4 and latency 4:

```
struct SharedOperands {
    a : u16
    b : u16
    c : u16
    d : u16
}

module SharedMultiply {
    clock clk reset rst
    in input : rv<SharedOperands>
    out output : rv<u33>

    input -> output {
        transform pipeline(auto) {
            implement {
                input.payload.a * input.payload.b
                    + input.payload.c * input.payload.d
                intent {
                    latency <= 4
                    ii == 4
                    dsp <= 1
                    minimize dsp
```

```
                }
            }
        }
    }
}
```

Only one transaction is active. While computation or an unconsumed result is present, new input is blocked. When `output.valid && output.ready` retires a result, the block may accept the next input on the same edge; this is why the `RETIRE_AND_RELOAD` schedule has II 4 rather than II 5. A stalled output remains valid with stable payload, and input readiness remains low until retirement is possible.

3.6 Frequency intent and physical timing

Frequency selection and the physical synthesis clock are separate controls:

Control	Purpose
`fmax >= 100` inside `intent`	Reject an implementation choice whose admitted frequency value is below 100 MHz.
`maximize fmax`	Rank the remaining legal choices by their admitted frequency values.
`fmax = { minimum = 100 }` in a selected project profile	Apply the same minimum to profile-selected regions.
`period-ns = 10.0` under the profile clock table	Emit a real 10 ns XDC/SDC clock constraint for downstream synthesis and place-and-route.
`evidence-policy`	Decide whether estimates are accepted, measurements are preferred, or measurements are required.

For a target of `F` MHz, the corresponding physical period is `1000 / F` ns. Setting only `fmax >= F` does not create a clock constraint and does not prove timing closure. Conversely, a clock period constrains the downstream tool but does not guarantee that the design meets it.

A project profile can keep these settings together:

```
[profiles.synth_trial]
backend = "systemverilog"
backend-mode = "required"
target = "xc7z030ffg676-1"
objective = "maximize fmax"
evidence-policy = "measured_preferred"
formal-policy = "available"

[profiles.synth_trial.constraints]
latency = { maximum = 4 }
ii = { exact = 1 }
fmax = { minimum = 100 }
dsp = { maximum = 8 }

[profiles.synth_trial.platform.clocks.clk]
period-ns = 10.0
```

Emit SystemVerilog and matching constraints from that profile:

```
zlang src/top.zhl --project zlang.toml --profile synth_trial \
  --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc \
  --constraints-sdc build/Top.sdc \
  --exploration-report build/implementation.txt
```

Supply `Top.sv` and the appropriate constraint file to the FPGA/ASIC synthesis flow. ZLang does not silently launch a vendor tool. For a bounded generic Yosys characterization trial, request both a report and cache:

```
zlang src/top.zhl --project zlang.toml --profile synth_trial \
  --systemverilog build/Top.sv \
  --synthesis-report build/yosys-synthesis.txt \
  --synthesis-cache build/yosys-cache \
  --synthesis-target generic-lut6
```

`estimate_only` uses estimates. `measured_preferred` ranks compatible routed measurements ahead of synthesis measurements and estimates, but still permits an estimate when no compatible measurement exists. `measured_required` rejects a choice when the required compatible measurement is absent. Measurements are specific to their target, part, tool, recipe, constraints, and design; a result from a different trial is not silently reused.

3.7 Current HLS boundary

This is not general HLS. It does not support overlapping transactions, modulo scheduling, arbitrary arithmetic graphs, more products, fixed-point temporal sharing, memories, CDC, or multiple active transactions. The transaction-stream formal route supplies bounded model-checking evidence for this exact case. `required_bmc` may accept it only after `bounded_pass`; `required_proven` rejects it because no unbounded proof is claimed.

The runnable example is `examples/temporal_shared_multiply.zhl`.

4 Simulation, waveforms, and verification

4.1 Native simulation

Native simulation runs a checked design without first generating SystemVerilog or starting an external HDL simulator. Use the Python API when a test needs direct input, state, and clock control:

```
from zlang import sim

program = sim.compile("examples/counter.zhl", top="Counter")
instance = program.create()
instance.tick("clk")
print(instance.get("y"))
instance.close()
```

4.2 Inputs, clocks, resets, and event schedules

For ordinary command-line use, choose either a bounded single-clock run or an explicit event schedule:

```
zlang sim examples/counter.zhl --top Counter \
  --clock clk --cycles 100 --json
zlang sim examples/multi_clock_stateful.zhl --top MultiClockStateful \
  --events events.jsonl --trace build/trace.fst
zlang sim examples/add.zhl --top Add --set a=200 --set b=55 \
  --compare-with iverilog --compare-artifacts build/sim-compare --json
```

Use `--set NAME=JSON` for ordinary typed binary inputs. Repeat it for multiple ports. `--cycles N --clock NAME` is a convenience for one-clock designs whose reset/input state is already suitable. Use `--events` for resettable designs, multiple clocks, coincident edges, or changing inputs. For example:

```
{"reset":{"rst":true},"edges":["clk"]}
{"reset":{"rst":false},"set":{"load":1,"value":37},"edges":["clk"]}
{"set":{"load":0},"edges":["clk"],"repeat":4}
```

`set` changes input values before the event. `reset` sets the named physical reset signal level. `edges` contains every clock edge committed atomically from one pre-edge snapshot; therefore coincident clocks belong in the same array. The reset spelling is the pin level, so an active-low reset is asserted with `false`. Synchronous reset takes effect on its domain edge, while supported asynchronous assertion takes effect in the event that changes the reset level.

Each non-empty `--events` JSONL line is one simulation event. An event may add `"repeat": N` to apply the same updates, resets and edges for `N` consecutive events; `N` must be a positive integer and the expanded schedule is bounded to 1,000,000 events. The CLI retains one parsed mapping and repeat count per JSONL record rather than copying its nested payload `N` times. This keeps long idle or backpressure intervals declarative without requiring a project-specific event generator.

4.3 U/X diagnostics

Ordinary mode uses deterministic binary-zero startup for state without an initializer. `--logic-state` instead initializes every unreset register bit to `U`, meaning it has never been refreshed. A write derived from unresolved state produces `X`; exact controlling values still resolve (`U | 1 == 1`, `X & 0 == 0`). This is hardware-debug information, not electrical `Z`, drive strength, or the Verilog event-region model.

Packed four-state spellings are MSB-first strings containing `0`, `1`, `u`, or `x`. Use `--set-logic NAME=01ux` for an input and `--initial-reg NAME=01ux` to override one named register path before the first event. The width must match exactly. `--strict-uninitialized` implies logic-state execution and fails when a typed observation requires an unresolved `U` or `X`; it does not alter compilation or Direct-SV.

Both modes execute the selected implementation. U/X tags describe diagnostic uncertainty, not additional physical hardware values. Legal exact binary optimizations can change U/X propagation, so the waveform is not a record of every operand in the original source spelling. Strict mode checks unresolved typed observations and stateful controls; resolved cancellation or controlling values need not fail. Traces and `get_logic()` retain tags for inspection.

4.4 Waveforms and differential RTL comparison

`--trace PATH.fst` writes the preferred compact native waveform; `--trace PATH.vcd` retains VCD interoperability with external and formal tools. The suffix selects the format. Other suffixes, including `.ghw`, are rejected. Physical clocks and resets are always present; repeat `--trace-signal NAME` to keep only selected data/state signals. The timeline uses two 1 ns ticks per simulation event and reflects the declared edge/reset contract. It is an event visualization, not an analog delay model. Standard FST and VCD both collapse unresolved provenance to `x`, so ZLang emits hidden `__zlang_meta/*_u_mask` companions that its query service combines back into distinct `U` and `X` values.

`--compare-with iverilog` and `--compare-with verilator` emit the normal direct SystemVerilog artifact, execute it in the selected external simulator, and compare every physical output after every event. A mismatch, X/Z value, missing sample, tool failure, or timeout fails closed. Comparison artifacts are retained only when an empty `--compare-artifacts` directory is supplied. Sequential RTL must receive the same explicit reset events needed by physical hardware; use `--events` when comparing a resettable design.

Exit status is nonzero for compilation/plan errors, native runtime failures, strict-U/X violations, verification/protocol failures, comparison mismatch, external tool failure, or timeout. Machine consumers should use `--json` and still treat the process status as authoritative. `--compare-artifacts` must name a new empty directory; without it, temporary comparison material is discarded.

One instance is not thread-safe; independent instances may run concurrently. `eval()` changes no architectural state. `edge_many()` computes all next-state values from one pre-edge snapshot and commits atomically. Synchronous resets act on their domain edge, while supported asynchronous assertion acts immediately. Values are exact-width and are never silently truncated.

There is no silent fallback to an RTL simulator. Native simulation requires the matching Linux x86-64/WSL2 wheel described in the installation chapter; an unsupported design or missing runtime produces an error.

4.5 Debug native simulation in VS Code

The Linux-x64 ZLang HDL extension debugs the same native simulation plan used by `zlang sim`. It does not generate stimulus. Give the debugger either the same JSONL event schedule used by the command line or a bounded single-clock `clock` plus `cycles` launch.

4.5.1 Debug without an event file

The declared `count = 0` supplies the initial state; this experiment does not assert reset. Clocks, resets, registers and memories are read-only in Variables. Use JSONL for explicit reset sequences, multiple clocks or repeatable

input changes. `events` and `clock`/`cycles` are mutually exclusive. The session ends after 20 clock events. Restart restores initial state rather than replaying manual edits.

For an interactive single-clock experiment, use the launch entry below:

1. Save `examples/rule_action.zhl`, select **RuleAction - manual inputs**, and press **F5**. The session stops before the first clock event.
2. Under **Variables > Ports**, right-click `enable`, choose **Set Value**, and enter `1`. Two **F10** steps produce `count = 1`, then `2` under **Registers**. Each step completes one hardware event, not one source line.
3. Set `enable` to `0` and step again: `count` holds at `2`.
4. Set `enable` back to `1`, use **F9** on `when enable` or its `count <- ...` action, then **F5 / Continue**. The next enabled event stops with `count = 3` and highlights the selected source line.
5. Open **ZLang HDL: Open Live Waveform** and keep **Follow Live** on to watch the same events. Input edits alone do not advance the clock; the waveform samples them when another event executes.

Add this entry to the `configurations` array in `.vscode/launch.json`, without an `events` field:

```
{
  "type": "zlang-hdl",
  "request": "launch",
  "name": "RuleAction - manual inputs",
  "program": "${workspaceFolder}/examples/rule_action.zhl",
  "top": "RuleAction",
  "clock": "clk",
  "cycles": 20,
  "logicState": true,
  "stopOnEntry": true,
  "traceSignals": ["enable", "fired", "count"]
}
```

4.5.2 Debug with a reproducible event schedule

Stable project vectors normally belong in `simulation/`. Temporary or generated experiments belong under ignored `build/` output. A project-specific generator may produce the JSONL file, but the debugger does not run an arbitrary generator as part of launch. This keeps a failing vector replayable from the terminal, VS Code, and native-versus-RTL comparison.

For example, save this schedule as `build/rule-action-events.jsonl`:

```
{"reset":{"rst":true},"set":{"enable":0},"edges":["clk"]}
{"reset":{"rst":false},"edges":["clk"]}
{"set":{"enable":1},"edges":["clk"],"repeat":2}
{"set":{"enable":0},"edges":["clk"]}
```

Then add a `.vscode/launch.json` configuration:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "zlang-hdl",
      "request": "launch",
      "name": "Debug RuleAction",
      "program": "${workspaceFolder}/examples/rule_action.zhl",
      "top": "RuleAction",
      "events": "${workspaceFolder}/build/rule-action-events.jsonl",
      "logicState": true,
      "strictUninitialized": false,
      "initialRegisters": {},
      "traceSignals": ["fired", "count"],
      "stopOnEntry": true
    }
  ]
```

```
}
```

Save the relevant `.zhl` files before launch. Source breakpoints are verified only on executable rules, FSM transitions, and verification clauses. One step executes one complete atomic simulation/reset/clock event. Continue runs until a verified source or data breakpoint, an exception, or the end of the bounded schedule. Registers and memories are read-only while paused; input ports may be changed. Assertion, protocol, strict-U/X, and runtime failures appear as debugger exceptions with source attribution.

Use **ZLang HDL: Open Live Waveform** for the active session. Signals listed in `traceSignals` have history from event zero; a signal added during execution is recorded from that point forward. Restart when complete history is needed. The readonly waveform editor also opens saved FST and VCD files independently of a debug session.

In the waveform view, click a row to select it. Change navigation and the Left/Right keys then follow transitions of that signal only. Mouse-wheel zooms around the pointer, Shift+wheel pans in time, and middle-button drag pans in time or scrolls the signal list. Waveform time retains the event-index meaning described above.

4.5.3 Python simulation without an event file

A Python testbench can supply inputs and clock/reset operations directly:

```
from zlang import sim

with sim.load("examples/rule_action.zhl", top="RuleAction", logic_state=True) as dut:
    dut.set("enable", 0)
    dut.reset("rst", asserted=True)
    dut.edge("clk")
    assert dut.get("count") == 0

    dut.reset("rst", asserted=False)
    dut.edge("clk")
    for enable, expected in [(1, 1), (1, 2), (0, 2)]:
        dut.set("enable", enable)
        dut.edge("clk")
        count = dut.get("count")
        print(f"enable={enable}, count={count}")
        assert count == expected
```

Save it as `tb_rule_action.py` in the repository root and run `.venv/bin/python tb_rule_action.py` there. The final three counts are `1`, `2`, `2`. For Python source breakpoints in VS Code, select this checkout's `.venv/bin/python` in the Python Debugger. F10 steps Python statements; `dut.edge("clk")` advances one hardware event.

This API creates a separate native instance, not a connection to the ZLang debug session. ZLang source breakpoints and **Open Live Waveform** do not follow a Python-driven instance. To use them, choose the manual-input launch above or replay a Python-generated JSONL schedule. The debugger does not run the Python testbench itself.

4.6 Source-level verification

Place goals in the source next to the hardware they describe:

```
module CheckedCounter {
    clock clk reset rst
    in enable : bit
    out value : u8

    reg count : u8 = 0
    step: when enable { count <- truncate<8>(count + 1) }
    value = count

    assert never_wraps_early @ clk { (count < 255) | !enable }
    cover reaches_ten @ clk { count == 10 }
}
```

Run all applicable goals for a selected top:

```
zlang design.zhl --top CheckedCounter --verify \
  --formal-jobs 4 \
  --verification-report build/verification.txt \
  --verification-work-dir build/verification-work
```

For a reproducible two-step flow, first publish a verification bundle and then run it:

```
zlang design.zhl --top CheckedCounter \
  --verification-bundle build/verify
zlang verify build/verify --mode bmc --depth 20 \
  --work-dir build/verify-work
```

The work directory contains solver logs and any counterexample or cover VCD. Keep it outside the bundle so reruns do not modify the input being checked.

4.7 Evidence terminology

Result terms have precise meanings:

- `bounded_pass`: no counterexample was found through the stated depth; this is not an unbounded proof;
- `proven`: the selected proof flow completed an unbounded proof;
- `witnessed`: a cover goal was reached and has a witness;
- `bounded_unreached`: a cover was not reached within the selected bound;
- `unknown`, `unavailable`, or `skipped`: no proof claim was established.

Each clocked goal names its clock with `@ clock_name`. Reset behavior follows the module's declared polarity, assertion mode, and release contract. Goals in independent supported domains run separately; this does not create a cross-domain temporal property. Use explicit assumptions only for real environment constraints, and never use an assumption to hide an invalid design.

Formal execution requires Yosys, SymbiYosys, `yosys-smtbmc`, and a supported solver such as Z3. The installation chapter shows how to check those tools. Examples are available in `examples/verification`.

5 Types and exact numerical semantics

ZLang types describe hardware representation exactly. Width, signedness, fixed-point scale, vector length, struct identity, and fixed-point overflow policy participate in typing and canonical identity.

5.1 Characters, fixed strings, and tuples

`char` is a source alias for canonical `u8`; it is intentionally not a separate nominal type. `string<N>` is a source alias for the corresponding fixed hardware collection and normalizes to canonical `vec<N,u8>`:

```
letter : char = 'A'
tag : string<4> = "OFDM"
joined = concat("802.", "11a")
```

Literals contain printable ASCII directly and may use `\0`, `\n`, `\r`, `\t`, `\\`, `\'`, `\"`, or an exact `\xNN` byte. Strings have no hidden terminator or length field and cannot be empty. They inherit vector indexing, equality, `length`, `generate`/`map`, homogeneous concatenation, pure functions, generic specialization, and typed compile-time constant parameters. Registers, FIFOs, synchronous memories, and ROMs store them with the same semantics as `vec<N,u8>`. When packed, the first character is the least-significant byte (`pack("AB") == 0x4241`). Unicode, interpolation, formatting, padding, and dynamic-length strings are not implicit operations.

Tuples are structural ordered aggregates. Tuple types use `(T,U)` and tuple values use the same positional spelling:

```
pair : (u8, bit) = (data, last)
payload = pair[0]
(next_data, next_last) = pair
```

Tuple arity is 2 through 8. Projection is a zero-based integer literal and flat destructuring introduces exhaustive immutable bindings. Nested tuple values are legal, but nested/wildcard patterns, runtime tuple selection, tuple update, arithmetic, and ordering are not. Exact `==`/`!=` is structural. Component zero occupies the least-significant packed region; `pack`/`bitcast` require every component to satisfy the ordinary non-enum packing rules.

`char` is indistinguishable from `u8` during overload resolution, and `string<N>` is indistinguishable from `vec<N,u8>`. Tuple identity, by contrast, is the recursive ordered sequence of component types.

5.2 Nominal enums

Enums give control state a nominal hardware type without exposing a numeric encoding in source:

```
enum State { Idle Header Payload Done }

reg state : State = State.Idle
active = state == State.Payload
```

Members use qualified names. Their declaration-order ordinals are encoded in `max(1, ceil_log2(member_count))` bits, so a one-member enum still occupies one bit. The encoding is a stable storage/backend ABI, but enum values are not integers: arithmetic, ordering, resizing, and cross-enum comparison are errors. Equality and inequality require the same nominal enum declaration.

When an external encoding is part of the hardware contract, declare an exact `bit`, `bits<W>`, or unsigned `uN`/`u<W>`/`uint<W>` representation. Omitted member values start at zero and otherwise continue from the preceding resolved value:

```
enum StatusCode : u8 {
    Idle
    Started = 10
    Waiting = 120
    Complete
}
```

This assigns `0`, `10`, `120`, and `121`. Every resolved code must be unique and fit the declared backing type; `enum Flag : bit { Clear Set }` is therefore limited to two values. Declaration order still controls exhaustive source selection, while the resolved code is the physical representation. The unbacked ordinal form above remains unchanged.

Raw boundaries use total, typed operations rather than enum casts:

```
raw     : bits<3> = enum_encode(rate)
valid   : bit = enum_valid<WifiRate>(raw)
decoded : WifiRate =
    enum_decode<WifiRate>(raw, WifiRate.Continue)
```

`enum_encode` is lossless and returns the enum's exact declared backing type (`bit`, `bits<W>`, or `uN`/`u<W>`/`uint<W>`); an unbacked ordinal enum retains its minimal `bits<W>` representation. `enum_valid<T>` and `enum_decode<T>` require that same exact representation. Decode returns its required same-enum fallback for every hole, so an invalid representation never becomes an invalid nominal value. These operations perform no resize.

Enum values remain nominal and cannot participate directly in arithmetic. An unsigned-backed enum deliberately exposes a numeric boundary through `enum_encode`, so `enum_encode(status) + 1` uses the normal widened unsigned arithmetic rules. A `bit`- or `bits<W>`-backed enum requires an explicit `bitcast<uW>` before arithmetic; there is no implicit raw-bits-to-integer cast.

Enums may appear in locals, outputs, registers/rules, structs, and vectors. A top-level enum input is rejected because an external invalid bit pattern would violate the nominal value invariant. General `unpack<Enum>` is likewise deferred. Exhaustive selection is described under operators and selection.

5.3 Nominal tagged unions

A tagged union represents one of several named payload shapes without exposing raw tag arithmetic:

```
union Message {
    Idle
    Data { value : u8 }
    Error { code : bits<4> }
}

message : Message = Message.Data { value = input }
result : u8 = match message {
    Message.Idle => 0
    Message.Data { value } => value
    Message.Error { code } => extend<8>(code)
}
```

The nominal declaration identity is part of the type. The source-order tag is stored in the most-significant bits; fields occupy the maximum-width payload in source order and shorter payloads receive zero low-order padding. A fieldless constructor is written `Message.Idle`. Every pure `match` is exhaustive, names each variant once, and binds exactly the declared fields. See the complete bounded surface.

5.4 Scalar families

Spelling	Meaning
`bit`	A one-bit control value, distinct from integers and raw bits.
`uN`, `u<N>`, `uint<N>`	An `N`-bit unsigned integer.
`sN`, `sint<N>`	An `N`-bit two's-complement signed integer.
`bits<N>`	An `N`-bit raw vector without integer arithmetic.
`fixed<W,F>`	Signed fixed point, `W` total bits and `F` fractional bits, wrapping target narrowing.
`ufixed<W,F>`	Unsigned fixed point with wrapping target narrowing.
`fixed_sat<W,F>`	Signed fixed point with saturating target narrowing.
`ufixed_sat<W,F>`	Unsigned fixed point with saturating target narrowing.

Widths are positive compile-time integers. Fixed-point types additionally require `0 <= F < W`. The families are distinct: no implicit signed/unsigned, integer/raw-bit, or integer/fixed conversion is performed.

Aliases disappear during typing:

```
type Address = uint<32>
type Payload = bits<64>
```

Alias cycles, duplicate declarations, and redefinition of built-ins are errors.

5.5 Integer width rules

An unsuffixed integer literal has the smallest exact hardware type implied by its value. Non-negative literals are unsigned; syntactic negative literals are signed two's-complement values:

Literal	Inferred type
`0`, `1`	`u1`
`2`, `3`	`u2`
`255`	`u8`
`-1`	`s1`
`-2`	`s2`
`-3`, `-4`	`s3`
`-123`	`s8`

Decimal, hexadecimal, and binary spellings with the same value infer the same type; leading zeros do not request storage width. A direct literal at an explicitly typed assignment, field, argument, or return boundary instead

uses that target type when the value fits exactly. An out-of-range literal is an error. Context never recursively resizes a compound expression, so `concat(0,1)` and `0 + 1` retain the types derived by their own operators.

Unary `-` applied to a non-literal remains ordinary typed arithmetic. It does not reinterpret an unsigned signal as a signed one.

Expression	Result
`uN + uM`	`u(max(N,M)+1)`
`sN + sM`	`s(max(N,M)+1)`
`uN - uM`	`u(max(N,M))`, modular underflow
`sN - sM`	`s(max(N,M)+1)`
same-family `N * M`	Width `N+M`
`&`, `|`, `^`	Matching family, widest operand width
shifts	The left operand's exact type and width
comparisons	`bit`

Addition and multiplication preserve carry bits. A left shift discards bits beyond its fixed result width. Signed right shift is arithmetic; unsigned and raw-bit right shift is logical.

Use explicit resizing:

```
wide = extend<17>(value)
low  = truncate<8>(wide)
```

`extend<N>` preserves signedness behavior and `truncate<N>` keeps low bits. Neither changes the type family.

At an explicitly typed output, local, register write, storage write, child binding, struct field, or declared function-return boundary, the width may be obtained from that boundary while the conversion remains explicit:

```
low : u8 = truncate(wide)
wide_again : u17 = extend(low)
```

The contextual and explicit-width forms normalize to the same typed resize. An inferred local, a nested arithmetic operand, or a mismatched type family has no such context and must keep `truncate<N>` or `extend<N>`.

5.6 Fixed point

Concrete spellings expose integer and fractional widths directly:

Spelling	Canonical type	Narrowing policy
`SF8.8`	`fixed<16,8>`	wrap
`SF_Sat8.8`	`fixed_sat<16,8>`	saturate
`UF8.8`	`ufixed<16,8>`	wrap
`UF_Sat8.8`	`ufixed_sat<16,8>`	saturate

For signed values the integer part includes the sign bit. `SF8.8` therefore represents `-128.0` through `127.99609375` in steps of `2^-8`.

Arithmetic is widened before any storage conversion. Fixed addition and subtraction require equal fractional widths; multiplication returns total width `W1+W2` and fractional width `F1+F2`. Target overflow policy applies only at an explicit or contextual storage conversion, not after every intermediate node.

Direct fixed-point literals must be exactly representable and in range, even for a saturating target. Loss is explicit:

```
y = quantize<SF_Sat8.8>(value) {
    round nearest_even
    overflow saturate
}
```

The rounding mode is mandatory; there is no implicit rounding default.

`round` mode	Discarded fractional part
`toward_zero`	Truncate toward zero.
`floor`	Round toward negative infinity.
`away_zero`	Round away from zero if any fractional part remains.
`nearest_even`	Round to nearest; on an exact tie choose the even stored integer.

Form	Target and overflow behavior
`quantize(value, MODE)`	Target comes from an unambiguous fixed-point assignment/return context; overflow defaults to that target type's policy.
`quantize(value) { round MODE overflow POLICY }`	Contextual target; both clauses are required.
`quantize<T>(value) { round MODE overflow POLICY }`	Explicit fixed-point target; both clauses are required.

`POLICY` is `wrap` (modular truncation to the target bit width) or `saturate` (clamp to the target's minimum/maximum). Ordinary `fixed`/`ufixed` targets default to `wrap`; `fixed_sat`/`ufixed_sat` targets default to `saturate` only in the short two-argument form. The block's explicit policy takes precedence. Conversion order is exact rescale, rounding, then overflow. `fixed_raw(0x0C22)` creates a target-context raw pattern; `fixed_to_raw` exposes the stored integer.

An exact fixed-point `dot(a,b)` retains the full accumulator. If the destination reduces fractional precision, give one post-accumulation rounding mode:

```
out y : SF_Sat8.8
y = dot(coefficients, samples, floor)
```

5.7 Vectors and structs

`vec<N,T>` has a compile-time length:

```
in samples : vec<4,u8>
```

An explicitly typed boundary can use a non-empty vector literal or contextual replication:

```
pair          : vec<2,u8> = [left, right]
zero_values   : vec<4,u8> = repeat(0)
zero_values_2 : vec<4,u8> = repeat<4>(0)
```

Every element must have the exact contextual element type. `repeat(value)` requires the destination vector to provide its length; `repeat<N>(value)` makes the length explicit. Both forms retain ordinary vector sequence order and lower to the existing typed generation representation.

Constant indexes are checked during elaboration. Runtime reads are accepted only when conservative range analysis proves every encoded value in range; ZLang does not silently mask or clamp. A rule may update one range-proven element of a one-dimensional `reg vec<N,T>` atomically. Nested paths, multiple dynamic writes to the same register action group, and runtime-selected instances remain unsupported; see Sequential logic, rules, and storage.

Structs are nominal products:

```
struct Packet {
    data : u64
    last : bit
}

packet = Packet { data = payload last }
```

Constructors are complete and may use same-name field punning. An immutable update rebuilds the same nominal struct and preserves fields not named in the update:

```
completed = packet with { last = 1 }
```

Unknown or duplicate fields and values of the wrong exact field type are rejected. This is a pure value expression, not mutation. Recursive layouts are still rejected. Generic structs are covered in Expressions, functions, and generics.

Structural `==` and `!=` are available for exact matching struct and vector types. Comparison is recursive over every field/element and returns `bit`; it does not perform conversion, resizing, or nominal coercion.

An immutable nominal struct can also be destructured exhaustively:

```
Packet { data, last } = packet
```

Every declared field must appear exactly once, under its declared name, and the new immutable bindings may not shadow another symbol. Destructuring lowers to ordinary typed field values; it does not add tuple patterns, partial patterns, renaming, mutation, or runtime matching.

5.8 Slicing, concatenation, and representation

An inclusive, compile-time bit slice returns raw bits:

```
in word : u16
out byte : bits<8>
byte = word[15:8]
```

Both `MSB` and `LSB` are compile-time integer expressions. The result is `bits<MSB-LSB+1>`. Reversed, negative, or out-of-range bounds are errors; slicing never resizes, reinterprets, or clamps its operand.

A `bits<N>` value also supports concise single-bit selection:

```
in raw : bits<24>
out lsb : bit = raw[0]
out msb : bit = raw[23]
```

Packed indices use conventional bit numbering: index zero is the least-significant bit, exactly as `[0:0]`. A runtime unsigned selector is legal only when its compiler-proven interval lies entirely within the packed width; there is no implicit modulo, clamp, or out-of-range hardware behavior. A fixed-width runtime window uses LSB-based `+:` spelling:

```
in raw    : bits<16>
in offset : u3
out byte  : bits<8>
byte = raw[offset +: 8]
```

`WIDTH` is a positive compile-time integer. The compiler requires `offset + WIDTH <= packed_width` for every possible offset. Vector element zero likewise occupies the least-significant packed element region.

`concat(a,b,...)` accepts at least two operands. When every operand is a vector with the exact same element type it returns one longer vector, preserving source and element order:

```
in left  : vec<2,u8>
in right : vec<3,u8>
out all  : vec<5,u8>
all = concat(left, right) // left[0], left[1], right[0], right[1], right[2]
```

Otherwise no operand may be a vector and all operands must be recursively bit-packable. The result is raw `bits<N>` and the first operand occupies the most-significant bits:

```
joined = concat(high, low) // high is the MSB portion
```

Each scalar operand is typed independently, before the result width is known. For example, `concat(0,1)` is `bits<2>` and `concat(3,0)` is `bits<3>`. Neither an assignment target nor a function caller may widen an operand or the completed concatenation implicitly.

Exact packed fill constants are available without manufacturing a numeric literal of the desired width:

```
clear_mask : bits<24> = zeros<24>
set_mask   : bits<24> = ones<24>
```

The width is a positive compile-time integer expression. Both forms produce exact `bits<N>` values and lower to ordinary constants; they are not vector replication. The separate `zero<x>`/`ones<x>` spellings inside an `equiv` declaration remain typed pattern constants rather than hardware expressions.

Vector/scalar mixtures and different vector element types are errors. Convert a vector explicitly with `bitcast<bits<W>>` when raw bit assembly is intended.

`reshape(value)` obtains a vector target shape from an explicitly typed context; `reshape<T>(value)` supplies it directly. Reshape recursively enumerates leaves outer-to-inner, then rebuilds the requested shape without packing them:

```
in matrix : vec<2,vec<4,u8>>
out flat  : vec<8,u8>
flat = reshape(matrix)
matrix_again = reshape<vec<2,vec<4,u8>>>(flat)
```

Source and target must be vectors with the same leaf count and exact same leaf type. Reshape has no endianness and performs no numerical conversion.

`bitcast<T>(value)` exposes or reconstructs the exact stored representation when source and target have equal packed width and both are recursively bit-packable and non-enum. It never resizes, sign-extends, truncates, rescales, rounds, or saturates:

```
raw    : bits<8> = bitcast<bits<8>>(signed_value)
signed : s8      = bitcast<s8>(raw)
```

`pack(value)` and `unpack<T>(raw)` remain supported low-level compatibility spellings and normalize to the same typed `Bitcast` operation. Signed integers and signed fixed-point values use their stored two's-complement pattern.

Aggregate layout is deterministic across simulation and generated RTL:

- struct field zero (declaration order) is at the MSB;
- vector element zero is at the LSB;
- tuple component `item0` is at the LSB;
- nested vectors and tuples apply the LSB-first rule recursively;
- nested structs retain declaration-order/MSB-first field layout.

The initialized `rom<T,N>` image format uses this same layout for each word. Images contain one exact-width binary word per line with address zero first; they do not inherit host byte order. Consequently simulator values and Direct-SV `$readmemb` agree on nested aggregate and fixed-point ROM contents. See Initialized synchronous ROMs.

Scalar numeric/raw types and recursively bit-packable structs, vectors, and structural tuples are accepted. Nominal enums, including an aggregate containing an enum, are intentionally rejected by `bitcast`, `pack`, and `unpack`. Enum ordinal storage remains available through ordinary typed values and `reshape`, but raw construction could create an invalid ordinal; no validity policy for that conversion is defined.

At explicitly typed assignment/storage boundaries, ZLang may insert an equal-width raw bitcast only when either source or target is `bits<N>` or flat `vec<N,bit>`. This makes raw wiring concise while preserving the rule that direct signed-to-unsigned, integer-to-fixed, resizing, and rescaling conversions are never implicit. Raw conversion does not participate in function arguments, operator overloads, generic inference, switch branch unification, or protocol compatibility.

Scalar `reduce(&, value)`, `reduce(|, value)`, and `reduce(^, value)` reduce the stored bits of `bit`, `bits<N>`, `uN`, or `sN` and return `bit`. `parity(value)` is the concise XOR form and also accepts `vec<N,bit>`. Existing vector reduction

still reduces elements and returns the element type; fixed-point and struct scalars are not representation-bit reduction operands.

6 Expressions, functions, generics, and compile-time hardware

ZLang expressions are pure typed hardware values unless they explicitly contain a timing construct. Assignment requires an exact canonical target type.

6.1 Operators and selection

From highest to lowest precedence:

1. calls, parentheses, aggregate selection, conversions, functional forms;
2. unary `-`, logical `!`, and bitwise `~`;
3. `*`, and compile-time `/` and integer `%` (left-associative);
4. `+`, binary `-`;
5. `<<`, `>>`;
6. `&`, then `^`, then `|`;
7. `==`, `!=`, `<`, `<=`, `>`, `>=`;
8. `&&`, then `||` in compile-time/equivalence guards;
9. right-associative `condition ? true_value : false_value`.

Unary `-` is available for numeric scalar/fixed types and may be overloaded for a nominal struct. It is not a general implicit-cast mechanism. Runtime logical `!` accepts exactly `bit` and returns `bit`; it is equivalent to an exact comparison with zero. `/`, `&&`, and `||` are available only where the compile-time/equivalence sublanguage accepts them; they do not infer runtime divider or general truthiness for multi-bit hardware values. Integer `%` is likewise compile-time only; it never infers a runtime remainder. Bitwise `~` accepts `bit`, raw bits, and signed/unsigned integers, preserving their exact type and width. It is not fixed-point negation.

Two-way runtime selection can use `?:` or `mux`:

```
y = select ? a : b
y = mux(select, a, b)
```

The condition is `bit` and both arms have one exact type. Numeric `switch` requires unique fitting keys and an `else` arm:

```
y = switch opcode {
    0 => a
    1 => b
    else => 0
}
```

A switch over a nominal enum uses qualified members and must cover every member. It needs no catch-all arm, which keeps a newly added state from silently taking an old default path:

```
next = switch state {
    State.Idle => State.Header
    State.Header => State.Payload
    State.Payload => State.Done
    State.Done => State.Idle
}
```

Numeric keys in an enum switch, members from another enum, duplicate members, and incomplete coverage are errors. Numeric switches retain their existing mandatory `else` behavior.

Nominal tagged unions use the parallel exhaustive `match` expression. Each arm names its union and variant and binds the exact source-declared payload names; there is no default arm or hidden priority:

```
y = match message {
    Message.Idle => 0
    Message.Data { value } => value
}
```

Typing lowers this surface to an ordinary exact-width `switch` over the union tag plus typed field projections before canonical optimization and backend emission.

Exact matching structs, vectors, and structural tuples also support `==` and `!=`. These comparisons recursively combine the ordinary exact leaf comparisons and return `bit`; mismatched nominal structs, vector lengths, tuple component order/arity, or element types are errors.

6.2 Pure functions

Functions are top-level, pure, and combinational. The smallest body is one result expression:

```
fn tap(sample : u8, coefficient : u8) -> u16 {
    sample * coefficient
}
```

They may call other non-recursive pure functions. They cannot capture module ports or contain state, clocks, delays, protocols, or pipelines. Return types may be explicit or inferred for both ordinary and generic functions:

```
fn make_tag(high : bits<2>) {
    concat(high, zeros<2>, ones<2>)
}
```

An inferred body is checked without a caller-provided expected type, and its exact final-expression type becomes the function signature. Forward references are resolved independent of declaration order. Direct or indirect inferred return cycles are rejected; ZLang does not attempt a recursive type fixed point. For ordinary functions, explicit and inferred spellings with the same exact signature lower to the same callable identity. Generic specialization identity continues to include the generic declaration form and its exact specialization arguments.

A body may introduce sequential immutable bindings without `let`, `const`, or `return`; the final expression is the result:

```
struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}
```

Each binding has its exact inferred type, may reference parameters and earlier bindings, and cannot shadow or be reassigned. Forward references and module captures are rejected. This syntax is normalized into the same concrete typed expression/call IR as the nested expression spelling.

Source units are declaration-order independent: functions, structs, enums, and modules may be declared after their users, and a shipped library unit may contain declarations without a hardware top. Parameterized modules can state compile-time obligations at specialization:

```
module Queue<type T,D=4>
where D >= 2 && is_power_of_two(D) {
    // ...
}
```

The constraint is checked against exact type/value arguments and creates no runtime hardware.

6.2.1 Typed constants and statically selected functions

Module and generic-function specializations may carry required named compile-time constants and pure function references:

```
fn widen(x : u8) -> u9 { extend<9>(x) }

fn transform_with<
    type A,
    type B,
    operation : fn(A) -> B
```

```
>(x : A) {
    operation(x)
}

module ImageBank<type T,N,IW,image : vec<N,T>> {
    clock clk
    reset rst
    in address : uint<IW>
    out data : T

    rom table : rom<T,N> { read_latency 1 init image }
    table.read_address = address
    data = table.read_data
}
```

These argument kinds are always named: `image=image` and `operation=fn widen`. A generic target may be selected explicitly, for example `operation=fn widen_as<A=u8,B=u9>`. Constants must have the exact declared, recursively bit-packable non-enum type and must evaluate completely during elaboration. A function reference must be pure and match the exact parameter and return types; it is not a runtime function pointer and cannot be stored, returned, compared, or carried through hardware ports.

Type and integer shape parameters resolve before constant/function parameters. The compiler retains a typed binding record containing the canonical constant value or exact callable identity, dependency closure, evaluator schema, and content digest. Source spelling is absent from semantic, artifact, and cache identity. Defaults for constant/function parameters are unsupported.

6.3 Compile-time integer modulo

Use `%` with integer constants, specialized value parameters and generated indices. It is accepted in generic value arguments, static indices and compile-time `if` conditions. For example, this pure function selects a chip from a constant PSS sequence; each specialization produces a constant `s2`:

```
fn nr_pss_chip<ID,K>() -> s2 {
    mseq = 0x42afa51b8fe1de592044c5d6c19a9cf6
    mseq[(K + 43 * ID) % 127] ? -1 : 1
}

module PssChip {
    out chip : s2 = nr_pss_chip<ID=2,K=126>()
}
```

Modulo uses floor-division semantics: a nonzero result has the divisor's sign. Thus `-5 % 3` is `1`, `5 % -3` is `-1`, and `-5 % -3` is `-2`. With a positive divisor `D`, the result lies between `0` and `D-1`, even for negative dividends. The arithmetic is compile-time integer arithmetic; any resulting width or index must still satisfy the destination's normal type and bounds checks.

An evaluated zero divisor is a source error. An unselected compile-time `if` branch is not evaluated. Both operands must be compile-time integers: input ports, registers and ordinary function arguments remain runtime values even with reset initializers or constant callers. Runtime and fixed-point remainder are unsupported; `%` does not infer a divider. Generated specializations remain subject to the usual elaboration limits.

6.4 Compile-time functions and selection

Compile-time evaluation is bounded and deterministic. Structural intrinsics are built in rather than stdlib overloads:

- `length(vector)`;
- `floor_log2(n)`, `ceil_log2(n)`, `index_width(n)`, `is_power_of_two(n)`;
- `pi()`, `sin(x)`, `cos(x)`, `log2(x)`, `log(base,x)`, `exp(x)`, and `sqrt(x)`.

Real intrinsics use a versioned decimal evaluator rather than host binary floating point. A non-integral real enters hardware only through explicit fixed-point quantization. `1 / sqrt(x)` is the compile-time reciprocal square root form used when generating seed or approximation tables.

`index_width(n)` is the storage width for an index into `n` elements: `max(1, ceil_log2(n))`. Its argument must be a positive compile-time integer expression. Integer parameter defaults are resolved in declaration order and may reference only earlier parameters:

```
module Table<N=8,IW=index_width(N)> {
    // Ports and implementation omitted.
}
```

A forward reference such as `module Bad<IW=index_width(N),N=8>` is rejected.

Specialization-time `if` accepts value parameters and canonical type parameters:

```
fn type_flag<type T>(x : T) {
    if T == u8 { 1 } else { 0 }
}

module Stage<D=1> {
    out y : u8
    if D == 1 { y = 1 } else { y = 2 }
}
```

Only the selected branch is semantically checked. Runtime signals are rejected in compile-time conditions with a suggestion to use `?:`, `mux`, `switch`, or a guarded action.

6.5 Functional datapath

Ranges are half-open and compile-time bounded:

```
products = generate(i in 0..8) a[i] * b[i]
doubled  = map(i in 0..8) { a[i] + a[i] }
y        = reduce(+, products)
z        = sum(i in 0..8) a[i] * b[i]
d        = dot(a, b)
```

`generate` and `map` produce `vec<N,T>`. `reduce` supports `+`, `*`, `&`, `|`, and `^`; `sum` is addition reduction. Reduction order is a deterministic source-order balanced tree, so finite-width intermediate types are stable across backends. `dot` uses full-width products followed by that same reduction.

For a nominal struct element, only `sum`/`reduce(+, ...)` is defined. Every internal node resolves the ordinary exact `operator +` overload for the two types produced by its children; the overload result is the exact type presented to the next node. Small and otherwise ineligible reductions retain that complete typed tree directly. Eligible large pure bounded regions instead retain a `FunctionalRegion` plus an `ExactReductionPlan`; the plan records the same balanced topology, exact intermediate result types, and resolved overload/callee identity without cloning every call body. These are two storage forms for one language contract, not two reduction semantics.

There is no component-wise field rule, implicit conversion, identity insertion, or special case for `Complex`. The frozen reduction tree is not reassociated, and explicit quantization remains exactly where the source or overload body placed it.

Range bounds may use resolved module value parameters. Runtime ranges, empty value-producing ranges, and unbounded generation are rejected.

Compile-time iterator arithmetic is mathematical integer arithmetic. It does not require hardware `extend`/`truncate` noise:

```
reversed = generate(i in 0..48) bits[47 - i]
```

For a vector, `values[first..past_last]` is a half-open compile-time range and lowers to the same ordered static element references as the corresponding vector literal. It is not a runtime slice and does not change packed bit-slice syntax `raw[MSB:LSB]`.

A functional iterator is a compile-time integer and may feed a generic value specialization directly or through a bounded constant expression:

```
fn coefficient<K>() -> u8 { K }
```

```
values = generate(k in 0..4) coefficient<K=k>()
next   = map(k in 0..4) { coefficient<K=k+1>() }
total  = sum(k in 0..4) coefficient<K=k>()
```

The evaluated value, not the binder spelling, participates in specialization identity. Runtime values are not specialization constants, and an iterator binder cannot shadow a module parameter.

6.5.1 Scalable bounded functional elaboration

Large compile-time `generate`/`map` regions with a binder-invariant result type may use the compact functional representation described above. The compiler still type-checks the region with the ordinary exact rules and counts its logical elements against the same bounded generation budget (currently at most 4096); it does not defer type checking to a backend or create runtime loops. Callable specializations remain ordinary monomorphic definitions, and each use retains its own source origin.

The accepted whole-vector IFFT64 reference exercises an outer 64-lane `generate`, 64 exact products per lane, nominal complex reduction, and one final quantization. N=64 is validated through semantic analysis, canonical round-trip, and the simulator; N=8/N=16 exercise Direct-SV and Verilator. This evidence does not claim full 4096-multiplier N=64 RTL or a production streaming architecture. See the 802.11a validation report for the measured boundary and the later streaming implementation.

A fully compile-time generated vector may initialize an immutable ROM without external source generation:

```
fn coefficients<N>() {
    generate(i in 0..N) truncate<8>(i * i)
}

rom table : rom<u8,N> {
    read_latency 1
    init coefficients<N=N>()
}
```

The complete transitive function/intrinsic dependency identity participates in the ROM and companion hashes. Compile-time functions still have no filesystem, network, process, clock, state, random, or environment access; only the compiler publishes a validated image. The initializer must resolve to exactly the declared vector shape and element type.

6.6 Generic structs, functions, and operators

Generic parameters have explicit kinds:

```
struct Pair<type T> { left:T right:T }

fn duplicate<type T>(x:T) { x + x }

module Use {
    in x : u8
    out y : u9
    y = duplicate(x)
}
```

Calls support exact call-site inference or explicit specialization. Generic struct constructors infer their specialization from fields where unambiguous. Every accepted call is monomorphized; there is no runtime dispatch.

A direct integer literal may use a concrete formal argument type after explicit generic arguments or non-literal arguments have fixed that type:

```
first  = same<T=u16>(1)
second = combine(1, existing_u16)
third  = combine(existing_u16, 1)
```

If a type is inferable only from literals, each literal first receives its own minimum-width exact type and normal exact unification applies. Argument order does not select a wider type. Compound expressions never receive this literal-only contextual treatment.

Libraries may overload binary `+`, `-`, `*` and unary `-` for nominal structs. Built-in scalar/fixed operations cannot be replaced. Resolution uses exact type unification, prefers an exact concrete overload over a generic one, performs no implicit conversion, and enforces nominal-owner coherence. Specialized bodies are retained once as deterministic monomorphic callable definitions. Each use site is a typed `Call` carrying the exact callee identity, signature, result type, and its own source origin. Analyses that require concrete arithmetic use the shared bounded call-expansion service; nominal reductions remain opaque to exact reduction rules. Direct-SV emits one `function automatic` per specialization rather than cloning the body at every call site.

`std.math.complex` is ordinary ZLang source defining `Complex<T>`, arithmetic operators, `Butterfly<S,D>`, and explicit component quantization. For example:

```
Complex<fixed<18,16>> * Complex<fixed<16,14>>
    -> Complex<fixed<35,30>>
```

The scale-30 result cannot be added to a scale-16 value until explicitly quantized. Its `complex_sum(values)` helper is a discoverability alias that returns the same typed `Reduce` as `sum(values)`; it adds no compiler behavior. Generics and nominal reductions do not imply rescaling, rounding, saturation, or casts.

Traits, runtime polymorphism, generic methods, function-name overloading, higher-order functions, recursion extensions, and implicit conversions are unsupported.

7 State, clocks, rules, FSMs, and pipelines

ZLang keeps combinational values, next-state actions, timing, storage, and physical clock ownership explicit in source. Synchronous active-high reset is the default. Each independently declared clock has one reset contract and may own ordinary state in the same module.

7.1 Clock and reset

Single-domain modules declare one pair:

```
clock clk
reset rst
```

Because newlines are ordinary whitespace, the compact adjacent spelling is equivalent and introduces no combined declaration or inferred domain:

```
clock clk reset rst
```

Clock and reset are declared together. Multiple domains require explicit reset, port, and state ownership wherever inference would be ambiguous:

```
clock source_clock
reset source_reset @source_clock

clock destination_clock
reset destination_reset @destination_clock

reg source_count : u32 @source_clock = 0
reg destination_count : u32 @destination_clock = 0

rule TickSource @source_clock when 1 {
    source_count <- truncate<32>(source_count + 1)
}

rule TickDestination @destination_clock when 1 {
    destination_count <- truncate<32>(destination_count + 2)
}
```

An unannotated stateful declaration remains concise in a single-clock module. In a multi-clock module its domain may be inferred from one unique destination or dynamic operand domain; otherwise analysis reports `ZL-DOMAIN-AMBIGUOUS`. Constants are domain-neutral. Combining dynamic values from different domains, including through an unannotated local, reports `ZL-DOMAIN-CROSSING`.

Rules and priorities are scheduled within a domain. Rules in unrelated domains do not acquire cross-clock mutual exclusion or atomicity. FSM-generated state and rules inherit the FSM annotation. Exact `pipeline(N) @clk { ... }`, FIFO, memory, ROM, and CSR state similarly retain one resolved owner; inserted pipeline and balancing registers never act as CDC.

Implicit clock-domain crossing is always rejected. See Hierarchy and protocols.

The recommended asynchronous spelling asserts immediately and releases only after two active clock edges:

```
clock clk
async reset arst_n @clk { polarity active_low }
```

Normal state transition resumes on the third edge after external deassertion. Use the externally synchronized release contract only when the external source already guarantees synchronous deassertion to this exact clock. It removes the internal 2FF while retaining asynchronous reset for state with a declared reset value. The low-level `reset ... { mode asynchronous ... }` spelling retains raw native deassertion for compatibility. Exact polarity, falling-edge behavior, hierarchical propagation, and unsupported formal/target routes are documented in the physical clock/reset contract.

7.2 Registers and next state

```
reg count : u8 = 0
count <- truncate<8>(count + 1)
```

The explicit multi-clock spelling places the annotation before the initializer:

```
reg count : u8 @datapath_clk = 0
```

Expressions read committed beginning-of-cycle state. All accepted `<-` updates become visible together at the active edge. A register without an accepted update holds. Reset restores the declared constant. Multiple unordered writers are rejected.

The reset value is optional. `reg retained : u8` has no generated reset branch or reset sensitivity; its hardware value is unspecified until written. Native two-state simulation starts it from a deterministic zero seed for tool parity, which is not a power-up or reset guarantee in generated RTL.

Use `zlang sim --logic-state` to opt into the per-bit simulation lattice `0`/`1`/`U`/`X`. State without an initializer begins `U` (never refreshed). An accepted state write converts unresolved `U` bits to `X` (refreshed but unknown), while a non-write holds `U`. Exact controlling values still resolve: `U | 1` and `X | 1` are `1`, while `U & 0` and `X & 0` are `0`. `--strict-uninitialized` implies logic-state execution and turns a typed read of either `U` or `X` into an error that identifies which state was observed.

Observations belong to the selected implementation. Exact binary optimization need not preserve the original expression's U/X propagation, and strict mode does not diagnose every cancelled operand in the source. U/X tags are not an algebraic hardware domain; unknown stateful guards or memory addresses still fail rather than choosing a deterministic branch or location.

Python callers may use `set_logic`, `get_logic`, `outputs_logic`, and `Program.create(initial_registers=...)`. Packed spellings are MSB-first, case-insensitive `0`, `1`, `u`, or `x`; a single character broadcasts to the full width. Register names use the design's state paths, including explicit child paths such as `child.counter`; ambiguous or non-register names fail before any state is changed. An edge that fails because of an unknown stateful control or native execution error rolls back logic state, event flags, and trace state atomically. CLI equivalents are `--set-logic` and `--initial-reg`. Standard FST/VCD represent both `U` and `X` as `x`; ZLang emits a same-width hidden companion `__zlang_meta/*_u_mask` whose set bits distinguish `U`. `--trace-signal` may be repeated to select the traced surface, and the initial sample is emitted at time zero. The waveform is an event-index visualization: each JSONL event occupies two 1 ns trace ticks, reset changes precede that event's selected clock edges, and falling-edge, active-low, and coincident-domain levels follow the compiled domain contract. It does not reconstruct analog time or simulator delta-cycle regions that are absent from the event schedule.

This is not full Verilog simulation: high impedance (`Z`), electrical resolution/strength, and Verilog event-region semantics are not modeled. Ordinary simulation remains deterministic two-state simulation, and generated Direct-SV is unchanged by these simulator options.

An ordinary output is a combinational wire. Output-visible state is declared explicitly with `out reg`:

```
out remaining : s32              // combinational wire
out reg accepted : s32 = 0       // resettable stored output
out reg retained : u8 @data_clk  // stored output with no reset branch
```

A registered output is one state resource and the public output port at the same time. It is readable inside the module, accepts `<-` updates, and holds when no accepted action updates it. Its optional initializer is its reset value; omitting it has the same unspecified-hardware-reset semantics as an ordinary register without an initializer. The type must be register-storable. Ready/valid, credit, packet, request/response and other protocol endpoints are not `out reg` ports; their individual protocol fields retain their existing ownership and flow-control rules.

ZLang `out reg` names actual architectural storage. It is not the historical Verilog/SystemVerilog `output reg` declaration category: a ZLang registered output has hold behavior, one explicit state owner, and the same clock/reset and atomic-update rules as an ordinary ZLang register.

Control registers may use nominal enums. Reset and next-state values must be members of the exact same declaration, and an exhaustive enum `switch` is the preferred finite-state-machine spelling:

```
enum Phase { Idle Active Done }
reg phase : Phase = Phase.Idle
phase <- switch phase {
    Phase.Idle => start ? Phase.Active : Phase.Idle
    Phase.Active => finish ? Phase.Done : Phase.Active
    Phase.Done => Phase.Idle
}
```

The semantic simulator and Direct-SV share the declaration-order ordinal encoding. The source-authored APB bridge in `stdlib/bus/apb.zhl` uses an enum for its internal phase without exposing it through the external bus ABI.

For control-oriented state, `fsm` is concise syntax for the same enum register and guarded atomic rules:

```
enum TxPhase { Idle Header Payload Done }

out reg remaining : u32 = 0

fsm phase = TxPhase.Idle {
    Idle {
        when start -> Header { remaining <- length }
    }
    Header { when header_sent -> Payload {} }
    Payload {
        priority {
            when abort -> Idle {}
            when last -> Done {}
        }
    }
    Done { -> Idle {} }
}
```

The qualified initial member in `fsm phase = TxPhase.Idle` infers the exact nominal enum type. The explicit compatibility spelling `fsm phase : TxPhase = Idle` remains accepted; neither form infers from guards, actions, or transition targets.

Every member appears exactly once. A state body is `hold`, one transition, or an explicit `priority` block; source order never creates hidden priority. The state update and transition actions form one ordinary atomic rule, reset restores the declared initial member, and direct writes to the generated state register are rejected. `fsm` is lowered before canonical/backend IR and does not add a second scheduler or runtime procedural `if`.

The production Wi-Fi `IeeeIFFTInputStrip` is a second source-authored witness: its `Data`/`Flush` FSM atomically starts and retires a 64-transfer zero-frame flush, holds under backpressure, and returns to `Data` after the last accepted sample. Mid-flush reset restores the declared initial state and discards the incomplete protocol epoch.

7.2.1 Runtime-indexed vector register updates

A one-dimensional vector register supports one range-proven runtime element write in an action group:

```
reg samples : vec<64,u8> = generate(i in 0..64) 0

when capture {
    samples[index] <- value
}
```

This is exactly `samples <- VectorUpdate(samples,index,value)` over the beginning-of-cycle snapshot. The unsigned index must be statically proven in range. A second dynamic or static write to the same vector register conflicts; nested paths, slices, runtime-selected instances, and automatic memory inference remain outside this bounded form.

A rule guard may establish the required bound through a simple unsigned comparison and conjunction:

```
when (index < 64) & capture {
    samples[index] <- value
}
```

That fact is scoped to this rule only. The bounded refinement understands these simple comparisons/conjunctions; `index <= 64`, disjunctions, and uses outside the guarded rule do not prove the required `0..63` range.

7.3 Delays and fixed pipelines

```
delayed = delay<2>(value)
piped = pipeline(3) { expression }
```

`delay<N>` adds exactly `N` zero-reset scalar stages. `pipeline(N)` is an exact visible-latency contract: for a supported pure scalar expression DAG, planning may place real computation boundaries within those `N` cycles and inserts the required reconvergent-path balancing. It never changes the requested latency or II, and `pipeline(1)` remains exactly one cycle. Unsupported nodes fail closed; the emitter does not fall back to an unreported whole-expression output delay. Outside a scheduled pipeline region, non-timeless operands at an operator must still be latency-aligned explicitly.

Compiler-selected latency/resource alternatives use `implement { expression intent { ... } }`. Scalar `pipeline(auto)` is retired; protocol `transform pipeline(auto, ...)` is a separate ready/valid construct.

7.4 Exact module timing contracts

A scalar datapath module may publish one exact contract shared by all of its wire outputs:

```
module TimedChild {
    clock clk
    reset rst
    in x : u8
    out y : u8

    timing {
        latency 4
        ii 1
    }

    y = pipeline(4) { x }
}
```

The contract describes observable behavior; it does not itself insert registers or authorize retiming. Constants are timeless, ordinary input-dependent logic is known at latency zero, and explicit `delay<N>`/`pipeline(N)` adds exactly `N`. A supported pipeline region may use its selected schedule to place and balance those internal cuts. Timeless values may join a known path; joins outside that graph still require equal known latency.

Registers, rules, FIFO/memory/ROM observations, protocols, and uncontracted child outputs are `unknown` under the current bounded contract. A positive latency requires exactly one clock/reset domain. Only `ii 1` is accepted. A contracted child adds its declared latency to the common known latency of its bound scalar inputs; an unaligned parent join is rejected instead of silently balanced.

This is distinct from `implement` constraints, target estimates, and measured evidence. A selected profile must preserve an exact module contract as immutable public behavior.

7.5 Guarded atomic actions

The concise action form is:

```
when increment {
    count <- truncate<8>(count + 1)
}
```

`<-` is exclusively a state update. It may target an ordinary register, an `out reg`, or a supported indexed vector-register element. An ordinary `out` wire is never stored and therefore cannot be updated with `<-`. A transient rule/FSM output uses `drive` instead:

```
out fired : bit

rule publish when accept {
    drive fired = 1
}
```

The driven wire is zero when no owning action fires. It participates in the same atomic scheduling, conflicts and explicit priority as the rule's other effects. `drive` cannot target a register or `out reg`, and a registered output cannot also have a combinational assignment. Because changing a wire update to stored state is a semantic choice, the compiler reports both alternatives but does not apply an automatic editor fix.

`drive` is a logical transient output assignment. It does not model a tri-state driver, high impedance (`Z`), electrical or pad drive strength, analog behavior, or Verilog multiple-driver resolution. Its zero default is therefore pulse/event semantics, not retained state.

Form	Cycle semantics	What it is not
`out value : T` with `value = expression`	Continuously defined combinational value	Stored state
`out reg value : T [@clock] [= reset]` with `value <- expression`	Clocked architectural state that holds when not updated	Verilog's historical output declaration category
`drive value = expression` in an action	Scheduled transient wire effect; zero when no owning action fires	Tri-state/electrical drive or implicit storage

Named compatibility spelling remains supported:

```
rule increment_count when increment {
    count <- truncate<8>(count + 1)
}
```

Action blocks may select effects recursively with runtime `when`, `else when`, and `else`:

```
fault_update: when fault {
    when valid_state {
        overflow_state <- 1
    } else {
        valid_state <- 1
```

```
        overflow_state <- 0
        sticky_state <- fault_code
    }
} else when clear_valid {
    valid_state <- 0
    overflow_state <- 0
} else when clear_overflow {
    overflow_state <- 0
}
```

An `else` binds to the nearest unmatched `when`, and an `else when` chain selects its first true predicate. A false `when` without an `else` contributes no effects; later statements in the enclosing action block are still considered, so multiple sibling conditionals can contribute to the same transaction.

Read this as two operations that never feed back into each other: predicates first choose the effects, then the scheduler either commits the complete chosen transaction or commits nothing. Storage readiness and rule conflicts belong only to the second operation; they can suppress a transaction but can never make an `else` branch run.

Every guard and operand reads the same pre-edge snapshot. The complete tree retains one outer `Rule`, one rule-fire identity, and one `ActionGroup`; selected effects and unconditional surrounding effects commit together or not at all. If no effect is active, the rule does not fire. Source order creates neither state visibility nor implicit priority, and reset suppresses the complete group.

Branch choice depends only on the predicates. If the selected branch contains an illegal FIFO or memory action, the whole action group is suppressed; it does not fall back to an `else` branch because another branch's storage action would be ready. The scheduler checks conflicts and storage legality using only active effects. Explicit `drive` output writes are scheduling resources too: a rule that loses a priority conflict on an output cannot commit its otherwise unrelated state effects.

Each runtime action condition must have exact type `bit`. Opposite structured branches may write the same resource, but writes on potentially overlapping paths are rejected; the compiler does not attempt arbitrary Boolean theorem proving. An explicitly empty branch is a valid no-op, while an entire action tree with no possible effect is rejected.

Runtime action selection is distinct from both other selection forms:

- compile-time `if` chooses declarations or values during elaboration and is not an action statement;
- `condition ? a : b`, `mux`, and `switch` select pure combinational values;
- runtime `when` selects effects that participate in one cycle-level atomic transaction.

Rules read beginning-of-cycle state. If the guard is true, all resource actions are legal, and scheduling permits the rule, its actions commit atomically. Reset suppresses all rules. Conflicting writers require explicit priority:

```
priority {
    clear: when clear_request { count <- 0 }
    increment: when increment_request {
        count <- truncate<8>(count + 1)
    }
}
```

Priority suppresses a lower enabled rule only when it conflicts with a fired higher rule. It is not procedural `if`/`else`: nonconflicting enabled rules may fire together.

A simple ordered chain may instead be written as:

```
priority first > second > third
```

This syntax expands to the adjacent edges `first > second` and `second > third`. Only declared edges participate; source order adds no priority. Use the block/pair form for any other priority graph.

7.6 Recommended asynchronous reset

The concise asynchronous form is safe for ordinary state. In a multi-clock module each annotated domain owns an independent conditioner:

```
clock clk
async reset arst @clk
```

It means asynchronous assertion followed by deassertion synchronized through two registers. Reset remains active on the first two active clock edges after the external pin is deasserted; normal state transition resumes on the third edge. Reassertion at any point immediately restarts that release sequence.

The external pin may be active-low:

```
clock clk
async reset arst_n @clk {
    polarity active_low
}
```

If reset deassertion is already synchronized to this exact clock outside the generated module, state that physical contract explicitly:

```
clock clk
async reset arst_n @clk {
    polarity active_low
    release externally_synchronized
}
```

This retains asynchronous assertion for resettable state but emits no internal two-register release conditioner. The annotation is never inferred: every destination clock domain must receive a release synchronized to that clock. Omitting `release` retains the safe internal 2FF default.

A register participates in reset only when it declares a reset value:

```
reg control : u8 = 0 // asynchronous reset to zero
reg datapath : u8    // no reset branch or reset sensitivity
```

The unreset register continues to accept enabled clocked updates while reset is asserted. Its RTL power-up value is unspecified until the design writes it. Native two-state simulation uses a deterministic zero startup seed, which is not a source-level power-up or reset guarantee. The opt-in logic-state mode represents unrefreshed bits as `U`; strict observation rejects `U` or computed `X` rather than collapsing either to a binary value. See Registers and next state.

A falling-edge clock uses falling edges for the two release cycles as well. Polarity affects the external pin and generated RTL; the simulator still accepts a logical `True` for asserted reset.

The lower-level compatibility spelling remains available for a raw native asynchronous assertion/deassertion contract:

```
clock clk { edge falling }
reset rst_n @clk {
    mode asynchronous
    polarity active_low
    power_up unspecified
}
```

This full block does not insert synchronized release. When either low-level physical block is present, every clause in that block is required. Raw active-low protocol helpers gate transfers with the deasserted high level; they do not reinterpret the physical pin as active-high.

8 Hierarchy, interfaces, protocols, and CDC

ZLang represents modules, endpoints, ownership, clock domains, connections, and physical instances explicitly before backend lowering. Connections are never inferred by guessing generated RTL names.

8.1 Modules and instances

Modules may have compile-time type and value parameters:

```
module Engine<type T, DEPTH=4> {
    in  value : T
    out copy  : T = value
}
```

Instantiate and bind scalar inputs explicitly or by same-name shorthand:

```
inst engine : Engine<T=u8,DEPTH=4> { value }
result = engine.copy
```

Specialization identity is distinct from physical instance identity. A single-domain parent may supply an unambiguous child clock/reset implicitly; multi-domain hierarchy requires explicit compatible domains. Child protocol endpoints use `connect`, not scalar binding syntax.

A reusable component fingerprint covers its typed hardware and the deterministic transitive closure of callables that hardware actually invokes. Functions that are merely visible through a parent/import context do not participate. Thus the same parameterless child reached through different parents is emitted once, while different value/type specializations or a different reachable helper body remain distinct and fail closed on an identity conflict.

When every intermediate child conforms to one exact named interface with one protocol input and one protocol output, an option-free path can be written as:

```
input -> decode -> execute -> output
```

This lowers to the ordinary pairwise typed edges `input -> decode.rx`, `decode.tx -> execute.rx`, and `execute.tx -> output`. Intermediate names are physical instances, not guessed ports. Arrays, unnamed or ambiguous signatures, edge options, adapters, buffering, and crossings remain explicit.

One-dimensional compile-time instance arrays are structurally unrolled:

```
inst lane[N] : Lane<W>
generate(i in 0..N) {
    lane[i].x = values[i]
}
outputs = generate(i in 0..N) lane[i].y
```

Structural generation may be nested and may contain compile-time `if`/`else`. Both are elaborated by the same bounded unroller; they do not create runtime loops or dynamic topology:

```
generate(i in 0..N) {
    generate(j in 0..N) {
        if i == j { lane[i].x = values[j] }
    }
}
```

The existing total expansion limit applies across the complete nested tree. Exceeding it reports a bounded compile-time-generation diagnostic rather than partially elaborating the design.

A runtime selector may project one exact bit-packable wire output. This is a read-only mux over the already elaborated children, never a dynamic physical instance:

```
selected = lane[select].output
```

Every `lane[i]` still exists, receives its compile-time bindings, and advances its own state independently. The selector neither routes child inputs nor gates a child clock, reset, rule, or storage action. The explicit equivalent is `outputs = generate(i in 0..N) lane[i].output` followed by `selected = outputs[select]`; both spellings retain the same physical child identities.

The bounded supported form covers:

- combinational scalar children, including exact aggregate scalar outputs;
- single-domain scalar children with registers/rules;

- primitive ready/valid children, including scalar wire ports beside the endpoint and the existing one-FIFO profile;
- legacy storage-only children owning one FIFO, synchronous memory, or initialized ROM;
- the scheduled-FIFO profile in which FIFO actions and ordinary register writes share one existing `ResolvedTransition`; and
- an outer one-dimensional array whose element contains a bounded same-domain scalar or direct-ready-valid hierarchy. Inner compile-time scalar arrays are retained as distinct physical paths rather than flattened by generated name.

The first-level request/response profile is also bounded and explicit. Each array child has exactly one `ordering in_order` endpoint plus scalar wire ports, and each connection names compile-time indexed requester and responder elements:

```
generate(i in 0..N) {
    requester[i].issue = issue[i]
    responder[i].accept = accept[i]
    connect requester[i].mem -> responder[i].mem
}
```

Every physical element retains its own outstanding ledger and reset epoch. Out-of-order matching, an additional protocol on the same child, nested or transitive request/response arrays, and unindexed endpoint references fail closed. This profile has an executable capability-registry witness and is covered by compiler, simulator, and backend validation.

Legacy globally controlled storage still cannot be combined with user registers/rules. Credit and other non-RV protocols, CSR or storage below a nested array element, CDC arrays, runtime-selected inputs or protocol endpoints, cross-module atomic scheduling, and incomplete bindings are rejected.

Concise declarations normalize to the same hierarchy IR when unambiguous:

```
frontend : AXI4LiteToRegBus<32,32>
axi : AXI4Lite<32,32>.slave @clk
axi -> frontend.axi
```

Explicit `inst` remains the escape hatch for namespace ambiguity.

8.1.1 Typed external modules

A bounded scalar-wire component may declare portable behavior with an ordinary pure ZLang model:

```
extern module VendorAdd : AddIfc { model add_model }
```

The concise source form is `extern module VendorAdd : AddIfc { model add_model }`.

The applied interface and model produce one exact semantic signature. Physical SystemVerilog is supplied separately through a hash-validated `ExternalPhysicalMapping`; inline HDL never defines ZLang semantics. The supported form has non-parameterized, clockless scalar inputs and one scalar output in Direct-SV. State, protocols, arrays, and generic external components fail closed.

8.2 Clock and reset boundary

Legacy `clock clk` plus synchronous active-high `reset rst` works through the production Direct-SV backend. A single-domain parent may supply that exact domain to a child only when the mapping is unambiguous; this is domain inheritance, not implicit CDC. Reusable stateful standard-bus cores use this same rule; source-authored pin adapters and concrete application tops remain the external domain owners. One non-default falling-edge, asynchronous, or active-low physical contract is typed and emitted by Direct-SV. Concise `async reset` adds one root-owned two-edge release conditioner and passes the conditioned reset through the closed child ABI; the raw asynchronous compatibility form remains distinct. Instance arrays require one compatible inherited domain unless they are purely combinational. Multiple independently conditioned asynchronous domains are supported in Direct-SV. Every domain of a stateful child must match exactly one parent physical clock/reset contract; no implicit reset crossing is inserted. Multi-domain asynchronous formal execution, power-on reset, and implicit reset crossing remain fail-closed.

8.3 Public top ABI

The selected top always exposes integration-friendly typed leaves. Struct fields and structural tuple components, including those inside protocol payloads, become individually named ports; tuple paths use `item0`, `item1`, and so on. A vector leaf remains one multidimensional packed SystemVerilog array; for example `vec<8,u8>` becomes `logic [7:0][7:0] samples`. It is neither an anonymous 64-bit public bus nor eight separately named ports. Nested `vec<Struct>` values become one array per struct field, and `vec<Tuple>` values become one array per tuple component.

Direct-SV RTL uses this public `TopPhysicalABI`. The conversion follows the canonical packing order with element zero in the least-significant region: for a descending packed vector dimension, source element zero maps to physical index `0`. There is no source annotation, compiler option, or backend-specific preprocessor branch selecting another public ABI.

8.4 Ready/valid

```
in  rx : rv<u8>
out tx : rv<u8>

tx.payload = rx.payload
tx.valid = rx.valid
rx.ready = tx.ready
```

For an input endpoint, payload/valid enter the module and ready leaves it. Output ownership is reversed. `.transfer` is a read-only protocol property meaning `valid & ready`. A source stalled with `valid=1` and `ready=0` must hold payload and valid stable.

Direct composition names source then sink:

```
connect rx -> tx
connect buffered_rx -> buffered_tx { buffer 2 }
```

A ready/valid buffer is real FIFO state; it does not appear implicitly.

8.5 Credit and request/response

`credit<T,N>` provides pulse-return flow control. `.transfer` is the physical send after credit gating, `.credits` is the current bounded count, and `.return` returns one credit. Counters begin at `N`.

Adapters are explicit:

```
connect rx -> tx { adapter rv_to_credit }
connect crx -> rtx { buffer 2 adapter credit_to_rv }
```

Request/response endpoints declare ordering and outstanding capacity:

```
interface mem : request_response<Request,Response> {
    max_outstanding 2
    ordering in_order
}
```

The requester owns request payload/valid and response ready; the responder owns request ready and response payload/valid. Request and response channels expose their own read-only `.transfer` events. Directional buffers are independent:

```
connect requester.mem -> responder.mem {
    request_buffer 4
    response_buffer 2
}
```

Buffered but unaccepted requests are distinct from accepted outstanding requests. The ledger increments on responder-side request transfer and decrements on requester-side response transfer. Reset begins a new protocol

epoch. `ordering out_of_order` requires a matching scalar ID field; hierarchical out-of-order composition remains outside the current bounded subset.

8.6 Aggregate protocols and the standard library

The compiler-shipped `std` namespace maps to ordinary `.zhl` files below `stdlib/`. The compiler knows generic aggregate schemas, roles, ownership, hierarchy, and domains; it does not hard-code AXI/APB transactions.

Available source-authored profiles include RegBus, AXI4-Lite, APB, AXI-Stream, Wishbone B4 Classic, bridges, and the CSR target:

```
import std.bus.reg
import std.bus.axi_lite

module AxiCsrTop {
    clock clk
    reset rst
    interface axi : AXI4Lite<32,32>.slave @clk
    inst frontend : AXI4LiteToRegBus<32,32>
    inst csr : RegBusCSRTarget<32,32>
    connect axi -> frontend.axi
    connect frontend.regbus -> csr.regbus
}
```

Aggregate pass-through connects each member according to typed ownership, including reverse ready:

```
input -> output
```

Protocol member properties such as `.transfer`, FIFO observations, and request/response channel events are part of typed semantics. They are not arbitrary user-defined fields and cannot be assigned when read-only.

8.7 Packets, virtual channels, and arbitration

`packet<T>` extends ready/valid with `.last`. Arbiters explicitly select source order, `fixed_priority` or `round_robin`, and `beat` or `packet` grant lifetime. `vc_credit<T,V,C>` maintains independent credit counts per virtual channel.

8.8 CSR interfaces

A `csr` block declares a typed register-map boundary with explicit access, reset, and bit placement. RW, read-only, W1C, pulse, reserved, constant, and alias fields retain their behavior through the standard RegBus and bus frontends. Overlap, incomplete placement, incompatible aliases, and unsupported clock ownership are errors. Generated Markdown or JSON register documentation comes from the same checked layout.

8.9 Clock-domain crossings

Implicit CDC is an error. Existing explicit forms are:

Crossing	Supported endpoint
`sync_level`	Slowly changing `bit` level
`pulse_toggle`	Rate-limited `bit` pulse
`handshake`	One scalar ready/valid payload
`async_fifo(N)`	Ready/valid stream through a power-of-two dual-clock FIFO

```
connect source -> destination { crossing async_fifo(4) }
```

The crossing is a semantic boundary. Its output has destination-domain provenance and may feed destination-domain combinational logic, rules, FSMs, or state normally. The compiler never rewrites, balances, or resource-packs logic through the boundary, and never chooses a crossing automatically.

```
connect enable_a -> enable_b { crossing sync_level }
rule Observe @clk_b when enable_b { seen <- 1 }
```

An aggregate with exactly one forward ready/valid member may use the same `async_fifo` crossing; its complete payload is atomic. Both domains must participate in a coordinated reset episode.

The physical Direct-SV FIFO uses one dedicated 1W1R `async_mem` with a one-cycle registered read, two-stage Gray-pointer synchronizers, and one prefetched output beat. A stalled beat holds both payload and `valid`; the read/consumed pointer advances only on transfer, so that beat still occupies capacity. Continuous accepted beats can transfer at II=1 without an extra output bubble. The synchronizers model digital CDC behavior, not analog metastability or an MTBF guarantee.

The ordinary `std.storage.core.StorageAsyncFifo<T,D>` wrapper exposes this same crossing as a reusable module with explicit writer and reader clock/reset ports. It lowers through the existing CDC IR; its module name is not special to the compiler. A normal `fifo<T,N>` never changes into an asynchronous FIFO because its endpoints happen to use different domains.

No automatic protocol adaptation or implicit CDC is performed. Full AXI4 bursts/IDs and bus-specific CDC bridges remain deferred.

Named module interfaces give a public name to an exact, behavior-free module signature. They let a library state the ABI that several concrete modules must implement without selecting, replacing, or instantiating any implementation.

```
interface FirIfc {
    clock clk
    reset rst
    in  x : fixed<18,16>
    out y : fixed<18,16>

    timing {
        latency 4
        ii 1
    }
}

module DirectFir : FirIfc {
    // The complete public surface is inherited from FirIfc.
    y = pipeline(4) { x }
}
```

The declaration after `interface` contains only public signature items:

- type and value parameters;
- `clock` and `reset` declarations;
- scalar, wire, ready/valid, credit, packet, and VC-credit `in`/`out` ports;
- source-authored aggregate protocol endpoints and their roles;
- at most one exact `timing { latency N ii 1 }` contract.

It cannot contain assignments, registers, rules, storage, child instances, or other implementation behavior.

8.10 Exact conformance

`module M : Ifc` means that `M` promises to have exactly the applied signature of `Ifc`. Conformance is checked during typed semantic analysis. It is not structural duck typing and does not introduce an implicit conversion.

The following all belong to the contract:

- parameter names, kinds, declaration order, and defaults;
- applied type and value parameter values after specialization;
- port names, declaration order, directions, canonical types, and protocol capacities;
- clock/reset pairs and every port or endpoint domain;
- aggregate protocol specialization, role, member ownership, payload types, and member domains;
- presence and exact value of the module timing contract.

If a conforming module declares no public surface, the complete applied interface surface is inherited: parameters, ports, clock/reset domains, aggregate endpoints, and timing. This is exact signature inheritance, not structural

inference. The legacy fully redeclared form remains accepted. A partial redeclaration is never merged with the interface and fails exact conformance.

Missing or additional members in a fully redeclared surface are errors. So are renamed ports, changed directions, width changes, role changes, a different domain, and a timing contract present on only one side.

Named parameters and concrete type parameters are supported:

```
interface TransformIfc<type T, N=4> {
    in  x : vec<N,T>
    out y : vec<N,T>
}

module Transform<type T, N=4> : TransformIfc<T=T, N=N> {
    y = x
}
```

The module parameter declaration itself must match the interface parameter declaration exactly. A concrete specialization records canonical applied values in `ModuleSignature`; concise source spelling does not change that identity.

9 Storage and memories

Storage declarations state logical behavior. A target mapping is legal only when it preserves the declared ports, latency, collision, mask, reset, and initialization contract.

9.1 FIFO

```
fifo queue : fifo<u8,4>
queue.data = rx.payload
queue.push = rx.transfer
queue.pop = tx.transfer
```

Read-only observations include `.front`, `.count`, `.full`, `.empty`, `.ready`, `.valid`, `.overflow`, and `.underflow`. A legal simultaneous pop/push while full preserves occupancy and ordering. An empty pop cannot consume a same-cycle push. Rules may instead own `queue.push(value)` and `queue.pop()` atomically; rule actions and globally driven controls cannot be mixed for one FIFO.

9.2 Synchronous memory

```
memory table : mem<u32,256> {
    read_latency 1
    collision write_first
}
table.read_address = read_address
table.write_enable = write_enable
table.write_address = write_address
table.write_data = write_data
read_data = table.read_data
```

An ordinary `mem` accepts `read_latency 0..16`: zero is combinational and each positive value is an exact number of read-domain edges from address capture to the visible result. Rule-owned memory reads have latency one and hold their result when no read fires. A target's synchronous RAM route may support a narrower subset; preferred policy can retain generic RTL, while required policy rejects a mismatch.

`collision old`, `new`, and `no_change` define a same-clock registered-read collision. `read_first` and `write_first` are aliases for `old` and `new`. Reset suppresses writes and actions.

9.3 Byte masks

An optional mask is exactly `bits<ceil(element_width / 8)>`. `write_mask[0]` controls the least-significant byte. The final lane contains only live bits when the element width is not byte-aligned; disabled lanes retain their prior values. A `write_first` read returns the post-mask merged word.

```
table.write_mask = write_mask
rule store when enable { table.write(address, data, byte_mask) }
```

9.4 Named same-clock ports

Up to eight named `read_port`, `write_port`, or `read_write_port` declarations may share one resolved clock domain:

```
memory table : mem<u32,1024> @clk {
    read_write_port a
    read_write_port b
    read_latency 1
    collision old
    write_priority a > b
}
```

Every writer appears exactly once in `write_priority`. Priority applies only to writes to the same address. Mixed port widths and asymmetric depths are errors. A matching target resource may implement the shape; bounded 1W+nR replication and small register-array fallback are available where documented. Banking and arbitration remain explicit source architecture.

9.5 Reset and initialization

`init VALUE` supplies one exact compile-time element value for every cell. With `contents clear`, reset writes the declared value in generic/register-array hardware. With `contents preserve`, it is a deterministic power-up value and runtime reset leaves cells unchanged. The optional reset block must specify both contents and read-data policy:

```
memory table : mem<u32,256> {
    read_latency 1
    collision old
    reset {
        contents preserve
        read_data preserve
    }
}
```

Without the block, both policies are `clear`. A target binding must advertise compatible initialization and reset behavior; inference in one tool run is not portable evidence.

9.6 Explicit dual-clock memory

`async_mem<T,N>` has one write-only port and one read-only port in different explicit domains:

```
memory table : async_mem<u32,1024> {
    write_port wr @write_clk
    read_port rd @read_clk
    read_latency 1
    collision old
    reset { contents preserve read_data clear }
}
```

Cells change on the writer edge; visible read data changes on the reader edge. The accepted read latency is `1..16`. Coincident logical edges use one pre-edge snapshot: `old` observes the previous cell and `new` forwards the coincident write. This is a digital contract, not a metastability guarantee. Stronger cross-clock collision claims require exact target evidence.

9.7 ROM

```
rom twiddles : rom<Complex<Twiddle>, N / 2> {
    read_latency 1
    init fft_twiddles<T=Twiddle,N=N>()
}
```

A ROM initializer evaluates at compile time to exactly `vec<depth,element_type>`. Reads are synchronous with one-cycle latency; reset clears the result register but never changes immutable contents. Direct-SV emits a deterministic companion image with address zero first and the canonical packed layout described in Chapter 5.

9.8 Mapping boundary

Memory elements may be recursively bit-packable non-enum aggregates as well as scalars. Full or partial writable images, automatic banking, unbounded multiport storage, mixed-width ports, ECC, and asynchronous 2RW memory are not part of the current language surface. Standard-library wrappers compose common shapes through these same memory contracts; their names do not grant special compiler semantics.

10 Standard library and buses

The standard library is ordinary tracked ZLang source under the logical `std` namespace. The compiler checks types, protocol roles, ownership, and domains; library modules own their transaction state machines. No bus name activates a hidden compiler dispatcher.

10.1 Streams, storage, math, and coding

Import	Main facilities
`std.math.fixed`	Exact fixed-point abs, min/max, clamp, MAC, and explicit quantization helpers
`std.math.complex`	Generic `Complex<T>`, butterfly, operators, and explicit complex quantization
`std.stream.core`	Ready/valid register slice, skid buffer, FIFO, mux, and demux
`std.stream.serialization`	Bounded vector serializer and raw-bit collector
`std.dsp.fft`	Exact butterfly, bit reversal, and compile-time twiddle generation
`std.storage.core`	Delay, FIFO, reorder, ping-pong, memory, async-memory, async-FIFO, and ROM wrappers
`std.coding.core`	Parity, bit reversal, checked LFSR, and convolution helpers

These declarations retain normal language semantics. Complex multiplication keeps full product width and scale; callers place quantization explicitly. Stream routing propagates backpressure only through the selected static path. Storage wrappers lower to the memory contracts in Chapter 9 and do not grant implicit CDC or target mapping.

10.2 Bus profiles

Import	Supported profile
`std.bus.reg`	In-order RegBus request/response and CSR target
`std.bus.axi_lite`	AXI4-Lite with independent AW/W buffering
`std.bus.apb`	APB setup/access sequencing
`std.bus.ahb_lite`	Single-manager, aligned full-width AHB-Lite to RegBus
`std.bus.wishbone`	Wishbone B4 Classic single-beat with ACK/ERR/STALL
`std.bus.axi_stream`	Data/keep/strb/last ready/valid stream
`std.bus.axi_burst`	No-ID, single-outstanding, full-width incrementing bursts
`std.bus.axi4`	Bounded five-channel AXI4 with IDs and optional USER payloads

Reusable stateful bus cores inherit one exact enclosing clock/reset contract. Concrete application tops and flat-pin adapters own physical pins. Ambiguous multi-domain inheritance, missing direct-top ownership, and incompatible connected state owners are errors.

10.3 CSR and RegBus

`RegBus<32,32>` is the common typed CSR boundary. `RegBusCSRTarget` provides RW, sticky W1C, and one-cycle pulse fields; responses remain stable until transfer. AXI4-Lite, APB, AHB-Lite, and Wishbone frontends translate their bounded profiles to RegBus without changing error completion semantics.

10.4 AXI4-Lite, APB, AHB-Lite, and Wishbone

AXI4-Lite buffers AW and W independently and joins them only after both transfers. APB implements setup/access phases and holds controls while waiting for `PREADY`. Wishbone uses mutually exclusive normal `ACK` and abnormal `ERR` termination.

AHB-Lite preserves the pipelined address/data relationship: an accepted address/control phase is followed by write data in the next data phase. `HREADYOUT` remains low while the RegBus operation is pending. Local geometry errors and RegBus failures use the standard two-cycle ERROR response. The current profile accepts aligned, full-width transfers for power-of-two byte-addressable widths from 8 through 1024; subword transfers are outside the profile.

10.5 AXI-Stream

`AXIStream<DW>` carries byte-lane `data`, `keep`, `strb`, and `last`. `AXIStreamOf<T>` transports one typed semantic payload per transfer; it does not define an external TDATA serialization for every aggregate. Register-slice, skid-buffer, FIFO, mux, and demux components preserve payload stability under stall.

10.6 Bounded burst and AXI4 profiles

`AXI4BurstReader` and `AXI4BurstWriter` permit one aligned 1–256 beat transaction. AR/AW payloads are snapshotted until accepted, channel backpressure is independent, and counted framing checks RLAST or generates WLAST. Nonzero RRESP/BRESP sets the transaction's error result. The profile omits IDs, WSTRB, burst kind, cache/protection/QoS/region/user fields, multiple outstanding transactions, fences, CDC, and implicit adaptation.

The broader `std.bus.axi4` profile supplies five channels, bounded outstanding slots, IDs, bursts, and optional USER fields. It remains a bounded library profile rather than a claim of complete support for every AXI4 endpoint and interconnect feature. Exclusive commit uses its separate reservation contract; the ordinary subordinate does not emit `EXOKAY`.

10.7 Imports and current boundaries

Imports resolve transitively from compiler-shipped sources and pinned project dependencies; cycles and unsafe paths are rejected. Ordinary compilation does not fetch dependencies. Automatic protocol adapters, implicit CDC, AXI-Stream ID/dest/user, Wishbone burst/retry, AHB arbitration/SPLIT/RETRY, and arbitrary full AXI4 topology are not added by the library.

Runnable integrations include AXI4-Lite CSR, APB CSR, AHB-Lite CSR, Wishbone CSR, and the streaming packet engine.

11 Projects, dependencies, and reproducibility

ZLang source imports are logical names. Physical paths, Git URLs, and revisions belong to project metadata, never to `.zhl` source:

```
import acme.dsp.filters
import std.math.complex
```

An import may introduce a source-local qualifier without changing the logical module or dependency identity:

```
import std.math.complex as cx
```

```
in sample : cx.Complex<u8>
out total : cx.Complex<u9>
total = cx.complex_sum([sample, sample])
```

The qualifier applies to types, functions, and struct constructors declared by that exact logical module (`cx.Complex { re = ... im = ... }` is valid). It is erased before semantic typing, so the qualified and historical unqualified spellings have the same canonical identity. An aliased import does not expose those declaration names unqualified and does not re-export declarations from a transitive dependency. Aliases are not runtime values or filesystem names; wildcards, member renaming, and re-export remain unsupported.

The compiler-shipped `std` namespace keeps its existing resolver. Other package names are resolved only when the source belongs to a locked project.

11.1 Project manifest

A project is rooted by a versioned `zlang.toml`:

```
schema = 1

[project]
name = "demo"
version = "0.1.0"
source-root = "src"

[dependencies]
acme = { path = "../acme-dsp" }

[dependencies.bus_models]
git = "https://example.invalid/hardware/bus-models.git"
rev = "0123456789abcdef0123456789abcdef01234567"
```

The `schema` field is required project metadata: it tells `zlang lock update` how to interpret the manifest. Use the value shown for the current project format.

Imports use logical package and module names rather than absolute checkout or cache paths. A file `filters.zhl` directly below package `acme`'s source root is imported as `acme.filters`; nested directories append dotted components.

Path locators are relative to the manifest containing them. Git revisions must be full lowercase 40- or 64-digit hexadecimal object IDs. Branch names, tags, and abbreviated revisions are deliberately rejected because they are mutable.

11.2 Lock update

Dependency resolution and fetching are explicit:

```
zlang lock update --project zlang.toml
```

The command validates the complete transitive graph before publishing a deterministic `zlang.lock`. It records each package manifest, exact source-module index, imports, content digests, and dependency edges. Git content is populated in the project cache during this command. Path dependencies remain at their manifest-relative location and are checked byte-for-byte against the lock.

Resolution rejects:

- dependency cycles and conflicting package/module declarations;
- source-root traversal or symlink escape;
- a package whose declared identity does not match its dependency key;
- missing, added, removed, or modified locked `.zhl` modules;
- modified dependency-resolution fields in dependency manifests (profile-only edits are intentionally outside the resolution identity);
- missing Git cache content or a revision mismatch.

The lock is written only after the entire graph validates. Failed updates leave the previously accepted lock usable.

Scenario	What to do	Without it
Standalone source or compiler-shipped `std.*` imports	Compile directly; no project lock is needed.	No consequence.
Project with local path packages	Run `zlang lock update` after declaring dependencies and whenever their source or resolution fields change. Keep `zlang.lock` with the project.	Compilation rejects a missing or stale lock; it never silently uses changed dependency files.
Project with pinned Git packages	Run `zlang lock update` while Git/network access is available; retain the exact revision and local cache for later offline builds.	Ordinary compilation does not fetch; missing cache or changed revision fails.
CI or another machine	Supply the project manifest, its lock, exact path sources and cached Git revisions (or update the lock before the offline build).	A lock file alone does not provide Git source bytes; compilation fails closed.

`zlang lock update` resolves dependencies; it is not a package installer or a step required for every single-file invocation.

Bounded scalar `extern module` implementations may also be declared under `[external-mappings.NAME]` and selected by a profile's `external-mappings = ["NAME"]`. The lock records every HDL source path and SHA-256. Paths are project-relative, must remain inside the project, and may not be symlinks. The mapping changes generated SystemVerilog without changing the behavior expressed by the ZLang source.

11.3 Offline compilation

Ordinary compilation never fetches dependencies and never updates project metadata:

```
zlang src/top.zhl --project zlang.toml --check
zlang src/top.zhl --project zlang.toml --systemverilog build/top.sv
```

`--project` may name a manifest or its directory. Without it, `zlang` searches the source file's parent directories for `zlang.toml`. If no project is found, single-file and compiler-shipped `std.*` compilation retain their existing behavior; arbitrary external imports remain an explicit error.

Compilation checks the current manifest, lock, every path dependency, and every cached Git dependency before checking the source. It is read-only: unavailable or dirty content is a lock mismatch, not an invitation to fetch or rewrite files. Build outputs and ZLang output/cache directories must remain outside the resolved root, manifest, lock, dependency, and stdlib inputs.

12 Profiles, targets, and physical implementation

Implementation profiles keep physical policy outside portable `.zhl` source. They are named tables in `zlang.toml` and are selected explicitly:

```
[profiles.release]
backend = "systemverilog"
backend-mode = "required"
target = "xc7z030ffg676-1"
allowed-transforms = ["pipeline", "dsp", "reduction"]
avoided-transforms = ["reassociate"]
objective = "maximize fmax"
architecture = "Xilinx7SymmetricDSPCascade"
architecture-mode = "preferred"
evidence-policy = "measured_required"
```

```
formal-policy = "required_bmc"

[profiles.release.constraints]
latency = { maximum = 8 }
ii = { exact = 1 }
fmax = { minimum = 100 }
dsp = { maximum = 8 }
```

`maximize fmax` prefers a legal choice with the strongest compatible frequency measurement available under the selected evidence policy. It does not guarantee the frequency achieved on an FPGA or ASIC. An estimate is never reported as measured routed timing.

Schematic use inside a project containing the shown `src/fir.zhl` and profile:

```
zlang src/fir.zhl --profile release --systemverilog build/Fir.sv
```

A profile may select pinned scalar external-module mappings with `external-mappings = ["vendor-add"]`. Mapping declarations live in the top-level `external-mappings` tables of `zlang.toml`; their source files and hashes are fixed by `zlang.lock`. They may change generated SystemVerilog but do not change the source-language behavior of the design.

Only the selected profile is parsed strictly. This permits a project to carry profiles for newer compilers without making unrelated builds fail. Selecting an unknown profile, selecting a profile without a project, or using an unknown key is an error. Profile data is excluded from dependency resolution and lock contents.

ZLang keeps functional behavior, implementation architecture, and physical target data separate. Explicit selection maps the ordinary functional symmetric FIR example to four DSP48E1 resources on `xc7z030ffg676-1`. Without an explicit selection, the same source continues through the generic Direct-SV path. Bounded automatic selection is described under target-aware planning.

12.1 Compiler-shipped source descriptions

Target data is ordinary compiler-shipped ZLang source:

- `std.target.generic` defines the resource-free generic target;
- `std.target.xilinx.series7` defines the bounded DSP48E1 capability;
- `std.target.xilinx.xc7z030` defines the device part, inventory, and cascade capacity;
- `std.arch.xilinx7_fir` defines the one manual symmetric-FIR template;
- `std.target.asic.sky130` describes the real `sky130_fd_sc_hd` standard-cell library boundary and proves that the core records are not FPGA- or Xilinx-specific.

The core parser and IR understand generic resources, typed ports, operation and width limits, register sites, dedicated links, inventories, targets, and architecture requirements. The names `DSP48E1`, `PCIN`, and `PCOUT`, and the physical SystemVerilog binding all originate in the library definitions - not in functional ZLang or semantic branches.

The SKY130 profile intentionally emits ordinary RTL for downstream Liberty-based mapping. It records the documented `dfxtp_1`, `dfrtp_1`, and `dlclkp_1` cell identities, but does not claim a hard multiplier, SRAM, PLL, fixed device inventory, timing closure, or silicon signoff. Those inputs are PDK/project/flow-specific and must be supplied by the implementation flow.

An exact source-level `timing { latency N ii 1 }` block is public module behavior, not target-selection policy. Generic implementation graphs publish their derived `timeless`/`known`/`unknown` timing class and are explicitly backend-independent. A selected physical graph carries its realization backend so a Direct-SV resource plan cannot be attributed to another physical implementation. Target selection must preserve any exact public module latency.

The bounded Series-7 profile describes a signed 25-bit pre-adder, signed 25x18 multiply, 43-bit product, 48-bit P/PCIN/PCOUT path, optional A/B/D/AD/M/C/P register sites, and an adjacent/same-group 48-bit dedicated cascade. The XC7Z030 instance records 400 DSP slices and four locally available cascade positions; this is deliberately not a complete floorplan model.

12.2 Manual selection

```
zlang examples/symmetric_fixed_fir.zhl \
  --target xc7z030ffg676-1 \
  --target-architecture Xilinx7SymmetricDSPCascade \
  --target-architecture-mode required \
  --systemverilog build/SymmetricFixedFIR.sv \
  --implementation-manifest build/SymmetricFixedFIR.manifest.json
```

`required` turns every target, shape, width, inventory, dedicated-link, or backend failure into a diagnostic. `preferred` returns the generic graph on a mapping failure. `generic` and omission of target options preserve the existing generic implementation. This command performs explicit selection; bounded automatic target-aware selection follows the model described in this chapter.

12.3 Boundaries

The generic Direct-SV implementation remains available when no primitive graph is selected. Existing semantic-reference equivalence can validate the semantic fixed-point region, but its reference emitter does not model vendor primitives. Physical evidence therefore consists of the exact semantic oracle, Verilator execution of the separate resource behavior model, and real Vivado synthesis/place/route of the primitive artifact. No primitive-level formal infrastructure was added.

The automatic planner covers only its documented symmetric-FIR and signed-product regions; general placement/graph covering, DSP48E2, Intel physical emission, automatic storage/clock-resource mapping, and new formal machinery remain outside the current bounded support.

The bounded planner is implemented for two exact fixed-point region families on `xc7z030ffg676-1`: the symmetric FIR cascade and ordered signed-product reductions used by complex multiply (`p0 - p1` and `p0 + p1`). It extends the existing architecture alternatives/exact reduction planning/pipeline scheduling/timing alignment/deterministic cost selection/bounded exploration path; it is not a second exploration engine.

12.4 User contract

```
result = implement {
    quantize<fixed<16,14>>(acc) {
        round nearest_even
        overflow saturate
    }
    intent { latency <= 8 ii == 1 fmax >= 100 }
}
```

`ii` and the compatibility spelling `throughput` normalize to one canonical II constraint. Fmax values are MHz. `latency<=N` is a bound. `latency==N` is an exact observable sample latency. `maximize fmax` is an evidence-ranked selection objective, not an achieved-frequency guarantee.

The target is build configuration, for example:

```
zlang examples/symmetric_fixed_fir_auto.zhl \
  --top SymmetricFixedFIRAuto \
  --target xc7z030ffg676-1 \
  --target-evidence-policy measured_required \
  --systemverilog build/SymmetricFixedFIRAuto.sv \
  --implementation-manifest build/SymmetricFixedFIRAuto.json \
  --pipeline-report build/SymmetricFixedFIRAuto.report
```

Functional source never names DSP48E1, MREG or PREG. With no target, or with `--target generic`, the ordinary generic candidate and backends remain active.

12.5 How target evidence affects selection

This topic belongs here only as a practical distinction: an estimated resource cost is not a measured FPGA result, and an estimated Fmax is not routed timing.

Evidence kind	What it establishes
`structural_estimate`	Compiler estimate from the selected structure; no vendor run.
`synthesis_measurement`	Result after synthesis, not placed/routed timing closure.
`routed_measurement`	Measured result after implementation/routing for matching target and constraints.

`measured_required` accepts only compatible routed evidence for an Fmax constraint. Structural latency/II remain authoritative and are not erased by that policy. Compatibility includes target/part, architecture, complete graph hash, pipeline configuration, backend, clock constraint and tool/version. `measured_preferred` ranks the strongest compatible evidence cohort before the numeric objective and falls back to estimates only when no legal stronger candidate remains. This prevents a coarse structural Fmax estimate from being compared as though it were an equally authoritative routed measurement.

Packaged routed evidence is bound to the matching part, constraints, physical graph, pipeline configuration, backend and tool recipe; a different graph's measurement cannot be reused as proof of this one. The optional `--implementation-manifest PATH` records the selected configuration, resource and timing decisions and their provenance for inspection or replay. Internal JSON schema numbers are compatibility details, not language features or knobs users should set.

12.6 Current boundary

Automatic target planning is intentionally limited to the symmetric-FIR and signed-product Direct-SV regions above. BRAM, PLL/MMCM, Intel physical planning, arbitrary graph covering, II-changing sharing, and automatic fixed transformations are not enabled. Generic Direct-SV remains the fallback implementation. Existing formal infrastructure is unchanged and no primitive-level proof claim is made.

12.7 Clock constraints

A selected project profile can declare one explicitly named clock period:

```
[profiles.release.platform.clocks.clk]
period-ns = 10.0
```

The clock name must match the selected top's clock declaration. The period is a physical build setting and is not inferred from an `fmax` intent.

The CLI writes constraints only together with exactly one SystemVerilog output. The following command assumes a project-local `top.zhl` and the `release` profile shown above:

```
zlang top.zhl --profile release --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc --constraints-sdc build/Top.sdc
```

XDC and SDC initially contain only:

```
create_clock -name clk -period 10 [get_ports {clk}]
```

The clock edge and reset mode must match the source declaration. A stale or mismatched constraint request is rejected. XDC/SDC output is deterministic and is written atomically without following symlinks.

No reset false path is emitted: asynchronous-reset recovery/removal timing must not be hidden. This constraint publication supports only the exact single-clock profile shown here. Multiple clock domains are supported in language semantics and Direct-SV emission, but generated clocks, multi-clock XDC/SDC publication, pin/-package/IO properties, input/output delays, CDC constraints, and formal properties are not emitted by this facility.

13 Generate and integrate RTL

Write the selected top to an explicit file:

```
mkdir -p build
zlang design.zhl --top Top --systemverilog build/Top.sv
```

The command writes only the requested artifact and keeps stdout empty. Diagnostics and `--verbose` progress messages go to stderr. A bare `zlang design.zhl --top Top` invocation writes SystemVerilog to stdout, which is convenient for a pipe but less suitable for a reproducible project script.

Use `--check` when no RTL should be created:

```
zlang design.zhl --top Top --check
```

Lint the generated module with Verilator:

```
verilator --lint-only --top-module Top build/Top.sv
```

Project profiles can select a target and clock constraint:

```
zlang top.zhl --profile release \
  --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc \
  --constraints-sdc build/Top.sdc
```

Generated RTL preserves declared widths, signedness, reset modes, hierarchy, and protocol behavior. Unsupported combinations stop with a diagnostic rather than emitting a partial module. Public port names are stable; private helper names are not a source-language interface and should not be referenced by user code.

Use `--systemverilog` for file output or the bare command for stdout.

The backend consumes typed semantic IR. It never dispatches on AXI, APB, or RegBus module names: source-authored standard-bus components use the same aggregate endpoint, hierarchy, register, rule, and connection lowering as user modules.

Ordinary scalar register/rule components share the standalone/composed dispatch and register contributor. Independent rules do not require global scheduler enumeration merely because a module is instantiated as a child. Hierarchy validation is cached only within one emission; protocol/storage effects and conflicting multi-effect rules keep their existing exact paths. Generated private names now use a shared hierarchy-local allocation plan: `cfg_ready`, `lane_0`, `lane_0_value`, `result_pipe_s1`, and short specialized definitions such as `Counter_s1a2b3c4d`. Source names win over generated helpers; actual collisions receive a deterministic suffix. Public top/port ABI and source register names are unchanged, including existing public aggregate-leaf separators. BackendArtifact records the naming schema and complete semantic identities separately from regenerated RTL/VPI locators. Naming does not change circuit semantics or imply QoR gains.

Formal-only `rule.fire` now uses the same effective reset and polarity as production state, including the two-edge synchronized release and conditioned child reset. Real RTL covers both polarities, both active edges and nested hierarchy. Its separate global-guard enumeration scalability limitation remains: large independent-rule formal projections remain outside the scalable supported subset.

Fixed-point ports and expressions use the canonical scaled-integer types behind `fixed`/`ufixed`, concise `SF`/`UF`, and `_Sat` formats. Same-scale target narrowing and explicit rescaling are both emitted from `FixedConvert`; the backend never infers wrap, saturation, or rounding from source spelling.

13.1 Supported composition subset

- scalar and packed struct/vector/tuple datapaths, `char`/`string<N>` aliases, immutable locals, muxes, field-/projection access, range-proven runtime packed-bit indexing/fixed-width packed slices and runtime vector indexing with LSB-zero semantics, exact-width `bitcast`, homogeneous vector `concat`, compile-time `reshape` , reductions, fixed-point arithmetic/conversion, and selected or nested fixed-latency scalar pipelines with II=1;
- registers, next-state assignments, atomic rules, FSM lowering, reset, and priority in one or more explicitly owned domains, including one independent two-register asynchronous-assert/synchronous-release conditioner per applicable domain;
- reusable parameter specializations, distinct scalar physical child instances, and compile-time indexed combinational or single-domain sequential instance arrays, including aggregate scalar outputs and scheduled FIFO state combined with ordinary child registers/rules;
- mixed scalar/state/ready-valid/request-response children;
- mixed scalar plus ready/valid instance-array children, bounded nested same-domain scalar/direct-ready-valid array hierarchy, and first-level indexed in-order request/response requester/responder arrays;

- read-only runtime projection of one exact bit-packable scalar/aggregate wire output from an instance array; every physical child remains independently instantiated and active;
- the bounded single-input/single-output ready/valid `transform pipeline(auto)` with one pure pipeline scheduling product-reduction kernel and a compiler-owned global stall;
- ready/valid connections and finite connection FIFOs, including simultaneous push/pop and backpressure;
- scalar ready/valid `async_fifo(N)` clock crossings and aggregate crossings whose schema has exactly one forward ready/valid member; aggregate payloads cross as one packed atomic value;
- vendor-neutral scalar `sync_level`, `pulse_toggle`, and one-entry ready/valid `handshake` crossings, using the same coordinated-reset and latency semantics as the simulator;
- synchronous memories with the currently typed collision/read-latency model;
- fixed-priority and round-robin packet arbiters with beat- or packet-scoped grants;
- in-order hierarchical request/response channels with independent directional buffers as represented in the checked design;
- one mandatory public top ABI derived from `TopPhysicalABI`: structs are recursively exposed as named field leaves, tuples as deterministic `itemN` leaves, while every `vec<N,T>` leaf is a multidimensional packed SystemVerilog array (`logic [N-1:0][W-1:0] name` for a `W`-bit element); flat packed aggregate representations exist only as deterministic aliases inside the selected top or inside compact child components;
- source-authored AXI4-Lite, APB, AXI-Stream, Wishbone, RegBus, and CSR hierarchy used by the current standard-library and real-design examples;
- hardware-connected CSR `ro`, `pulse`, and sticky-W1C behavior with the frozen software/hardware priority policy;
- ready/valid-to-credit, credit-to-ready/valid, and per-VC credit source state machines from typed protocol/capacity metadata;
- recursive instance/state locators and exact physical-domain contracts for non-default reset modes. Specialization identity is not used as physical instance identity.

SystemVerilog reserved words are deterministically prefixed with `zlang_`. The collision check runs after that physical-name mapping, so two distinct semantic leaves can never silently become one RTL port. Inline-boundary helper signals are allocated outside the public namespace and remain deterministic. Packed dimensions use conventional descending ranges. ZLang element zero maps directly to physical packed index zero and the least-significant private slice. The generated artifact has one unconditional top definition and is accepted by both Yosys and Verilator without backend preprocessor branches. Unresolved locals are eliminated before backend lowering. Unsupported IR shapes raise `SystemVerilogEmissionError`; no artifact is published after a failed emission.

Ordered comparisons preserve typed signedness explicitly: both operands of `<`, `<=`, `>`, and `>=` are rendered through `$signed` or `$unsigned` according to the checked design, independent of whether an operand is a port, local, projection, or register. Equality and inequality remain raw bit comparisons.

Use the stable CLI option:

```
zlang examples/simple_dma.zhl --top SimpleDMA \
  --systemverilog build/SimpleDMA.sv
verilator --lint-only --top-module SimpleDMA build/SimpleDMA.sv
```

`--systemverilog` is an explicit artifact sink: it writes only the requested SV file and leaves stdout empty. A bare `zlang SOURCE` invocation emits the production Direct-SV artifact to stdout. Diagnostics remain on stderr.

Use `--verbose` when an explicit success confirmation is useful:

```
zlang design.zhl --systemverilog design.sv --verbose
```

This keeps stdout artifact-safe and reports the selected top and written paths on stderr. To validate parsing, top selection, and semantic analysis without publishing any backend artifact, use:

```
zlang design.zhl --check
```

`--check` prints a one-line success result and returns zero. Without `--top` it checks every module declared in the source file, preventing an invalid dormant module from being hidden by default-top selection. With `--top`, it checks only that selected elaboration root. Syntax, semantic, and top-selection failures retain their normal nonzero status and diagnostic. As a check-only mode, it cannot be combined with artifact output options.

13.2 Explicitly unsupported

Legacy globally controlled storage combined with user state, out-of-order or nested request/response arrays, additional protocols on a request/response array child, arbitrary nested storage/CSR/aggregate protocol hierarchy, CDC arrays, and runtime-selected inputs, protocols, or actions remain fail-closed. The backend never silently emits only one element or reconstructs an indexed connection from an RTL name.

Sequential ready/valid modules and role-qualified standalone in-order `request_response` requester/responder modules use the same typed clock/reset, ownership, and accounting semantics as their hierarchical forms. A standalone endpoint owns a local unbuffered ledger: request transfer increments it, response transfer decrements it, and reset starts a new empty epoch. Stateful generic child templates that still require parent specialization remain validated through their concrete parents.

13.3 Example and structural coverage

The release checks every standalone-supported example root through Direct-SV emission and strict Verilator lint. Child/template roots require a concrete specialization or parent. These tests are evidence for the shipped examples, not a language allow-list; unsupported new combinations fail closed. Current corpus counts and tool versions are published in release status.

The streaming FFT and IEEE 802.11a transmitter examples exercise typed hierarchy, ROMs, fixed-point datapaths, protocols and state. Their RTL is built through the same generic backend path as other designs, without module-name dispatch. Selected examples also have independent cycle-level oracle checks.

Generated RTL uses exact packed widths and single-driver combinational assignments. Procedural combinational blocks are used where defaults or branching require them; strict lint treats width, latch, driver and structural warnings as failures. A target binding is claimed only when the selected physical capability proves its clock, latency, collision and reset contract.

14 Installation and external tools

ZLang HDL supports CPython `>=3.12,<3.13`. The source compiler is release-tested on Linux x86-64. Native-simulation wheels are supplied for Linux x86-64 (including WSL2). macOS native wheels are not part of this release. You do not need to find a distribution package for that exact Python runtime: the recommended `uv` workflow can download and manage it independently of the system Python. Parsing, semantic checking and Direct-SV generation need only the Python package. Verilator, Yosys, SymbiYosys and a solver are external programs used only when their corresponding lint, synthesis or formal flow is requested.

The exact versions used by release acceptance are recorded in `release/status.json`. Other versions may work, but they are not evidence for the published release.

14.1 Recommended installation with uv

Install `uv` using its official platform instructions. Confirm that the executable is visible in the same terminal that will run ZLang:

```
uv --version
```

For ordinary use, install the wheel downloaded from the matching GitHub release as an isolated command-line tool. `uv` provisions the requested interpreter if it is not already present:

```
uv tool install --python '>=3.12,<3.13' \
  --with /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
zlang lock --version
zlang verify --version
```

If `uv` reports that its tool directory is not on `PATH`, run `uv tool update-shell`, then start a new shell. `uv tool dir --bin` prints the command directory for manual PATH configuration. To replace an installation made from a downloaded wheel, repeat `uv tool install --force ...` with the new wheel.

For development from a Git checkout, keep the environment local to the repository:

```
git clone https://github.com/postoroniy/zlang-hdl.git
cd zlang-hdl
make venv
source .venv/bin/activate
zlang --version
```

`make venv` installs the checkout compiler and its matching reviewed native wheel into one local environment. On a supported host with VS Code and Node.js 22, it also installs the matching extension. Use `INSTALL_EDITOR=0` to leave the installed extension unchanged. Installing only `.[test]` does not install the native runtime and is not equivalent to this complete setup.

The commands below use `zlang` directly: it is on `PATH` after `uv tool install`, or after activating the checkout's virtual environment with `source .venv/bin/activate` in a Bash/Zsh terminal. Alternatively, in the checkout use `uv run --no-sync zlang ...` without activating it. Open a new terminal and activate the environment again when returning to the checkout.

14.2 Conventional venv and pip alternative

If a compatible Python is already available, use whatever executable name the host provides; the example deliberately avoids assuming a version-suffixed command:

```
python3 -c 'import sys; assert (3, 12) <= sys.version_info < (3, 13), sys.version'
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install \
  /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
```

For an editable development checkout, replace the two-wheel installation command with:

```
python -m pip install -e '.[test]'
```

This fallback needs a compatible interpreter and `venv` support from the host. ZLang should be installed with `uv` or an isolated virtual environment, leaving the operating system's own Python unchanged.

14.3 Windows through WSL2

The current release is tested as a Linux application, not as a native Windows Python package. On Windows 10/11, install WSL2 with an Ubuntu distribution, open the Ubuntu shell, and perform the Linux installation entirely inside it. For example:

```
sudo apt-get update
sudo apt-get install -y curl git build-essential
```

Then install `uv` using its official Linux instructions and follow the `uv` steps above. Keep active projects under the WSL Linux filesystem rather than on a mounted Windows drive; this generally gives much better compiler, Git, test, and EDA-tool filesystem performance.

Inside a development checkout, these forms are equivalent:

```
source .venv/bin/activate
zlang examples/add.zhl --check
uv run --no-sync zlang examples/add.zhl --check
```

`uv run --no-sync` uses the already-created project environment without changing dependencies. Scripts should either activate the environment first or use `uv run --no-sync`, making them independent of interactive shell state.

14.4 Check and run ZLang

Check the installed commands:

```
zlang --version
zlang lock --version
zlang verify --version
```

14.5 Install the VS Code extension, LSP, and native debugger

The Python wheel installs `zlang-lsp`; the VSIX supplies the VS Code client, syntax support, and snippets. The Linux-x64 VSIX additionally contains the reviewed native debugger and waveform viewer. Install compiler/native wheels and the VSIX from the same release.

Verify the installed LSP command first:

```
zlang-lsp --version
```

Install the downloaded Linux-x64 package:

```
code --install-extension zlang-hdl-0.1.0-linux-x64.vsix --force
```

For a source checkout, `make venv` installs the checkout compiler, its matching reviewed native wheel, test tools, and `zlang-lsp` into `.venv`. Point VS Code at that exact server in `.vscode/settings.json`:

```
{
  "zlang.lsp.path": "${workspaceFolder}/.venv/bin/zlang-lsp"
}
```

Reload the VS Code window and inspect **View: Output -> ZLang HDL** if startup fails. The universal VSIX remains editor/LSP-only; use the Linux-x64 package when native debugging and the waveform viewer are required. The simulation chapter provides a complete launch configuration and JSONL stimulus example.

Check a design and emit production Direct-SV without installing any external EDA program:

```
zlang examples/add.zhl --check
mkdir -p build
zlang examples/add.zhl --systemverilog build/Add.sv
```

14.6 Choose which external tools you need

Only a compatible Python runtime and the `zlang-hdl` package are mandatory for checking source and emitting SystemVerilog. `uv` is an optional installer: it can obtain that Python runtime, but ZLang does not need `uv` after installation. Install other tools only for the workflows you actually use:

Workflow	Additional requirement	If it is missing
`zlang SOURCE --check` and SystemVerilog emission	None beyond Python and `zlang-hdl`	Neither Verilator nor Yosys is needed; a missing compiler package or incompatible Python prevents `zlang` from starting.
`zlang sim` (native simulation)	Matching `zlang-native-sim` wheel for the host	Simulation fails explicitly; compiler checking and RTL emission remain available.
Strict RTL lint or Verilator execution	`verilator`	RTL can still be emitted; lint and Verilator simulation cannot run.
Generic synthesis	`yosys`	RTL can still be emitted; synthesis cannot run.
Formal BMC, proof, or cover	`yosys`, `sby`, `yosys-smtbmc`, and one selected solver executable	Checking and RTL emission still work; requested formal jobs cannot complete.
Icarus-based test or simulation flow	`iverilog` and `vvp`	Only that selected flow is unavailable.
`zlang lock update` with Git dependencies	`git` and access to the pinned revision during update	Path-only projects and locked offline compilation still work; fetching a Git dependency does not.

`yosys-smtbmc` is the Yosys SMT model-checking driver. It is the executable sometimes informally shortened to "BMC"; there is no ZLang dependency named `bmct`. The default recorded formal route invokes the external `z3` executable through this driver. Bundle replay may instead select another supported SMTBMC solver, such as `boolector`, `bitwuzla`, `yices`, or `cvc5`, with `zlang verify BUNDLE --solver NAME`. Installing only a Python solver module is not a substitute for putting the corresponding executable on `PATH`.

14.7 Recommended: OSS CAD Suite

The simplest reproducible route is the official YosysHQ OSS CAD Suite. Its release archive provides Yosys, SymbiYosys, solvers and related open-source EDA tools together. Download the archive for the host platform from the OSS CAD Suite releases, extract it, then either source its environment file for each shell:

```
source /absolute/path/to/oss-cad-suite/environment
```

or add its `bin` directory to `PATH`:

```
export PATH="/absolute/path/to/oss-cad-suite/bin:$PATH"
```

Use an absolute installation path and keep the suite intact because Yosys and SymbiYosys also use adjacent data and support files. See the upstream OSS CAD Suite release page linked above for supported archives and platform details.

Install Verilator separately when it is not supplied by the selected suite. A distribution package is convenient:

```
sudo apt-get update
sudo apt-get install verilator
```

Distribution versions vary. Follow the upstream Verilator installation guide when the release-tested version or a source build is required.

14.8 Alternative: install components separately

Component installation is useful for distribution packaging or when a pinned tool is maintained independently:

- install Yosys using the upstream Yosys build/install instructions;
- install SymbiYosys using the upstream SymbiYosys documentation;
- install the `z3` command from the official Z3 releases or a suitable distribution package;
- install Verilator using its official guide linked above;
- install Icarus Verilog from the host distribution when the selected test or simulation flow requires `iverilog` /`vvp`.

Keep all selected tool `bin` directories on the same `PATH` used to start ZLang, the terminal, CI job or VS Code. Mixed host/container installations are a common reason for a tool being visible interactively but unavailable to the compiler.

14.9 Verify the installation

```
verilator --version
yosys -V
sby --version
command -v yosys-smtbmc
z3 --version
boolector --version
bitwuzla --version
iverilog -V
```

The release-tested external-tool versions are recorded only in the release status under `eda_toolchain`; the supported Python range is recorded under `platform.python`. These are evidence for that release, not universal tool-version requirements. Other versions may work; validate them for the flow you intend to use. Finding an executable on `PATH` is not, by itself, evidence that a formal or synthesis result is valid.

Run a real formal smoke test:

```
zlang examples/verification/bounded_counter.zhl \
  --verify --verify-require proven \
  --formal-depth 16 --formal-timeout 45 \
  --verification-bundle build/verify/counter \
  --verification-work-dir build/verify-work/counter \
  --verification-report build/verify/counter-result.txt
```

Expected exit status is zero. The `capacity` assertion should be `proven`, the `reaches_capacity` cover should be `witnessed`, and `exceeds_capacity` should be `bounded_unreached`. A bounded cover miss is not a proof of unreachability. See the runnable formal examples for a deliberate counterexample, scoped assumptions, immutable bundle replay and ready/valid checking.

14.10 Troubleshoot formal runs

Formal execution is opt-in. `--check`, normal compilation and SystemVerilog emission do not invoke a solver. `bounded_pass` is bounded evidence and is never reported as `proven`; unavailable tools or bindings remain explicit `unknown`/`skipped` according to the requested policy.

If a tool is not found, run the version commands above in the same environment that starts ZLang. If a proof is `unknown`, inspect the retained report and solver logs under `--verification-work-dir`; increasing depth or timeout does not turn an unsupported binding or reset model into an executable property.

15 Current limitations

This chapter collects the main boundaries of the current release. Earlier chapters describe the supported subset at the point of use.

15.1 Language and scheduling

- Implementation exploration is bounded and does not claim a global optimum.
- Ordinary scalar implementations are fixed-rate II=1. Automatic II>1 sharing is limited to the documented capacity-one ready/valid `a*b + c*d` form.
- There is no general HLS scheduler, modulo scheduling, arbitrary transaction overlap, or hidden retiming.

15.2 Simulation and verification

- Native four-state diagnostics model `0`, `1`, `U`, and `X`, not electrical `Z`, drive strength, or the Verilog event-region model.
- FST/VCD traces use an event-index timeline rather than analog delay timing.
- Bounded model checking yields `bounded_pass`, not `proven`. Formal support is property- and structure-dependent; unavailable routes establish no proof claim.

15.3 CDC, storage, and physical targets

- Clock-domain crossings require explicit supported crossing constructs. The compiler does not infer CDC intent.
- Memory latency, collision, masking, and port semantics must match a supported declaration and target mapping.
- Frequency intent filters evidence-backed candidates but does not itself create a clock constraint or prove timing closure.
- Generated XDC/SDC support is deliberately narrower than language-level multi-clock semantics.

15.4 Protocol and library boundaries

- Protocol conversion is explicit; no ready/valid, credit, request/response, or bus adapter is inserted silently.
- Standard buses are bounded typed library facilities. Full arbitrary AXI4, dynamic topology generation, and implicit external-IP binding are outside the current surface.

ZLang HDL rejects unsupported combinations instead of generating partial RTL. The following restrictions are part of the current user-facing contract.

15.5 Language and RTL boundaries

- ZLang has no runtime procedural `if`, mutable local variables, implicit numeric casts, implicit fixed-point rescaling, or general HLS scheduler.
- Integer `%` is compile-time only. Runtime and fixed-point remainder are unsupported; ports, registers and ordinary function arguments are not compile-time operands.
- Runtime-selected instance inputs/protocols, general cross-module atomic scheduling, full AXI4, automatic CDC insertion, and arbitrary stateful elastic pipelines are unsupported.
- Stateful objects inside one module may belong to independent explicit clock domains. A bounded `async_mem` is the sole storage exception: its one writer and one registered reader have distinct owners and form an explicit semantic boundary. Asynchronous 2RW memory, mixed widths, automatic banking, cross-clock atomic rules, derived/gated clocks, and target-aware scheduling across a CDC boundary are unsupported. Ordinary global memory control cannot be combined with unrelated user register/rule state in the same module; use hierarchy for that composition.
- Unsupported Direct-SV combinations produce a diagnostic rather than partial RTL.

15.6 Verification boundaries

Formal support is specific to each property family and observation boundary. `bounded_pass` is not an unbounded proof, and `skipped`, `unavailable`, and `unknown` make no proof claim. Report any accepted design that emits invalid RTL as a correctness defect.

A Appendix A — Concise source reference

This page is a concise guide to writing and running ZLang HDL. Source-language rules define the hardware; generated SystemVerilog is an output format, not a second language specification.

For an unfamiliar construct, consult the detailed language chapter and known limitations. For exact width tables and supported forms, follow the linked topic guide.

A.1 Non-negotiable rules

- Source files use `.zhl`. The old `.zl` suffix is rejected.
- Run `zlang SOURCE --check`; highlighting is lexical and is not validation.
- Native simulation needs a matching platform wheel and fails explicitly when it is unavailable.
- Hardware assignments are concurrent. `=` drives the current cycle and `<-` schedules next state at the active clock edge.
- Assignment types are exact. There is no implicit resize, signedness change, fixed-point rescale, protocol adapter, or clock-domain crossing.
- ZLang has no `let`, `const`, `var`, `return`, semicolons, C-style `input`/`output`, or runtime procedural `if` syntax.
- `if` is compile-time elaboration only. Use `?:`, `mux`, or `switch` for pure runtime values; use `when`/`else when`/`else`, `priority`, or `fsm` for state actions.
- Scalar `pipeline(auto)`, `architecture(auto)`, and `explore` are retired. Compiler-selected scalar implementation uses `implement`.
- `pipeline(N) { expression }` is exact N-cycle hardware, not an optimization request. Protocol `transform` `pipeline(auto, ...)` is a separate bounded ready/valid construct and remains supported.
- Unsupported combinations fail with a diagnostic. Generated signal names are not a source-language interface.

A.2 Smallest valid modules

Combinational logic:

```
module Add {
    in a, b : u8
    out y : u9 = a + b
}
```

Sequential logic:

```
module Accumulator {
    clock clk reset rst
    in enable : bit
    in value : u8
    out total : u9

    reg state : u9 = 0
    update: when enable {
        state <- truncate<9>(state + extend<9>(value))
    }
    total = state
}
```

All operands are read from one pre-edge snapshot. An accepted action group commits atomically; reset has priority. Multiple conflicting actions require an explicit `priority` relation or are rejected.

Multiple domains use the same declarations with one `@clock` owner:

```
module DualClock {
    clock a reset ra @a
    clock b reset rb @b
    reg left : u32 @a = 0
    reg right : u32 @b = 0
    rule TickLeft @a when 1 { left <- truncate<32>(left + 1) }
    rule TickRight @b when 1 { right <- truncate<32>(right + 2) }
}
```

An omitted owner is accepted only when exactly one domain can be inferred. Dynamic provenance follows through local expressions; an explicit `sync_level`, `pulse_toggle`, `handshake`, or `async_fifo(N)` is the only way to cross domains.

A.3 Types, literals, and exact conversions

Common scalar types are `bit`, `u8`, `s16`, `uint<N>`, `sint<N>`, and `bits<N>`. `char` is canonical `u8`; `string<N>` is canonical `vec<N,u8>`. Fixed point uses `fixed<W,F>`, `ufixed<W,F>`, or concise forms such as `SF8.8` and `SF_Sat8.8`.

- Unsuffixed integer literals use minimum exact width unless they occur directly at a concrete typed boundary.
- `a + b` and `a - b` follow the documented exact result-width rules; multiplication returns the full product width. Shifts preserve left width.
- Use `extend<N>` and `truncate<N>` for width changes.
- `quantize<T>` provides explicit fixed-point rounding/overflow. Its placement is part of the computation and cannot be moved into products or reductions.
- Compile-time `pi`, `sin`, `cos`, `log2`, `log`, `exp`, and `sqrt` generate constants only; non-integral results require explicit `quantize<T>`.
- Use `bitcast<T>` only for equal-width representation changes. `pack`/`unpack` are supported low-level compatibility forms.
- `concat(a,b,...)` places the first operand at the MSB. Destination context never changes operand widths.
- Packed bit index zero is the LSB. `x[MSB:LSB]` uses an inclusive static slice; vector ranges `values[first..past_last]` are half-open.
- Indexed aggregates are packed LSB-first: `vec`/`string` element zero and tuple `item0` occupy the least-significant component recursively. First-declared struct fields and tagged-union tags remain at the MSB.

See types and numerics before changing a width or a fixed-point type.

A.3.1 Operators at a glance

- Arithmetic: `+`, binary `-`, `*`, unary `-`. Built-in integer families do not mix signed and unsigned operands implicitly. For example, `u8 + u8` is `u9`, `u8 - u8` is modular `u8`, `s8 - s8` is `s9`, and `s8 * s8` is `s16`. Assignment does not narrow any of these results.
- Bitwise: `&`, `|`, `^`, `~`; logical `!` requires `bit`. Runtime Boolean control combines `bit` values with `&` and `|`, not `&&` and `||`.
- Shifts: `<<`, `>>` preserve the left operand's width and require an unsigned shift amount. Right shift of a signed integer is arithmetic.
- Comparisons: `==`, `!=`, `<`, `<=`, `>`, `>=` return `bit`. Ordering is numeric; raw bits and enums do not acquire integer ordering implicitly.
- Selection: `condition ? a : b`, `mux`, `switch`, and tagged-union `match`. Runtime conditions are `bit`; branches must have one exact result type.
- `/` and integer `%` are compile-time only. `&&` and `||` belong to the bounded compile-time/guard sublanguages, not runtime hardware truthiness.

See operator precedence and typing for mixed widths, fixed-point rules and overload resolution.

A.4 Aggregates and functions

```
struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}
```

Function bodies are pure and combinational. Bare body bindings are immutable, inferred, declaration-ordered locals; the final expression is the result. Return types may be explicit or inferred. Functions cannot capture module ports or contain state, protocols, delays, or pipelines.

Supported aggregates include `vec<N,T>`, nominal `struct`, nominal `enum`, tagged `union`, and structural tuples `(T,U)`. Prefer inferred locals when the type is exact and obvious. Keep explicit types at public ports, state, raw/enum boundaries, narrowing, and fixed-point quantization.

Struct updates use `value with { field = replacement }`, not a second struct constructor. Enum backing may be `bit`, `bits<W>` or an unsigned integer type. Omitted member codes start at zero or increment the preceding explicit code. Use `Enum.Member`, exhaustive `switch`, and `enum_encode` for a numeric boundary; raw encodings use `enum_valid<T>` and total `enum_decode<T>` with a same-enum fallback. A raw-bit encoding needs an explicit numeric `bitcast` before arithmetic.

Top-level enum inputs and arbitrary `unpack<Enum>` are not supported. Enum members such as `reserved`, `reset` and `priority` are valid contextual identifiers; lexical highlighting is not a reserved-name rule.

Generics are monomorphized. Type/value parameters and exact compile-time constraints are source declarations, not runtime hardware:

```
module Queue<type T,D=4> where D >= 2 && is_power_of_two(D) {
    // ...
}
```

Integer `%` is compile-time only: it may appear in generic arguments, static indices and compile-time `if`. It has the precedence of multiplication and division and uses floor modulo (`-5 % 3 == 1`, `5 % -3 == -1`). An evaluated zero divisor is an error. Ports, registers and ordinary function arguments are not compile-time operands; there is no runtime remainder.

Imports are logical. For example `import std.bus.reg` resolves compiler-shipped `stdlib/bus/reg.zhl`; `import std.math.complex as cx` adds a source-local qualifier but does not create a runtime namespace.

A.5 State, hierarchy, and protocols

- Declare one default synchronous domain with `clock clk reset rst`.
- `async reset arst @clk` means asynchronous assertion and fixed two-edge synchronized release. Reset modes are expressed by the clock/reset declarations rather than user logic.
- Add `release externally_synchronized` only when deassertion is already synchronized outside; no internal 2FF is emitted.
- `reg q:T = VALUE` resets to `VALUE`; `reg q:T` has no reset branch.
- `out q:T` is a combinational output wire.
- `out reg q:T [@clock] [= VALUE]` is real architectural state: it holds when not updated and follows register clock/reset scheduling. It is not Verilog's historical `output reg` declaration category.
- `drive q = expression` is a transient rule/FSM-owned logical wire effect, zero when no owning action fires. It is not tri-state, high-impedance, electrical-strength, pad, analog, or multiple-driver-resolution behavior.
- `reg`, `fifo`, `memory`, and `rom` retain explicit typed state/storage semantics. Memory latency, collision, mask, and reset policies are not inferred.
- A concise child declaration is `child : ChildModule`; `inst child : ChildModule` remains accepted. Bind only typed ports/endpoints.
- `rv<T>`, `credit<T,N>`, and `request_response` properties such as `.transfer` are typed observations, not ordinary struct fields.
- Use explicit `connect/source -> sink`, adapters, and CDC crossings. The compiler never inserts these silently.
- Top-level structs and tuples become recursively named leaf ports; vectors use multidimensional packed arrays in the production Direct-SV ABI.

Read sequential state/storage and hierarchy/protocols before composing stateful children.

A.6 Functional datapath and implementation intent

`generate`, `map`, `reduce`, `sum`, and `dot` are statically bounded functional hardware expressions. They are not runtime loops.

Common built-ins by purpose:

- Width/representation: `extend`, `truncate`, `bitcast`, `pack`, `unpack`, `concat`, `reshape`, `zeros<N>`, `ones<N>`, and `parity`.
- Fixed point: `quantize<T>`, `fixed_raw<T>`, `fixed_to_raw`, and the explicit `fixed_truncate_*` / `fixed_round_even_*` compatibility conversions.
- Enum boundaries: `enum_encode`, `enum_valid<T>`, `enum_decode<T>`.
- Compile-time shape: `length`, `floor_log2`, `ceil_log2`, `index_width`, `is_power_of_two`; compile-time numerical constants also use `pi`, `sin`, `cos`, `log2`, `log`, `exp`, and `sqrt`.
- Collections: `generate`, `map`, `reduce`, `sum`, `dot`, and `repeat`.
- Timing: `delay<N>(value)` and `pipeline(N) { value }` add exact cycles; they are not ordinary pure-function calls or architecture-search requests.

These are context-sensitive operations, not a promise of arbitrary operand types. Use the linked type/expression chapters for exact signatures. Imported stdlib helpers and user-defined functions are separate from compiler intrinsics.

Use exactly one `implement` region when the value is fixed and the compiler may choose among legal implementations:

```
y = implement {
    dot(samples, coefficients)
```

```
    intent {
        latency <= 4
        ii == 1
        dsp <= 8
        fmax >= 100
        minimize lut
    }
}
```

An Fmax constraint or objective compares available implementation choices. It is not a guarantee of achieved device or silicon frequency; estimates, synthesis measurements, and routed measurements are distinct.

The expression is the hardware meaning. `intent` contains hard constraints and one supported objective; it does not name optimization passes. Search is bounded and deterministic rather than an exhaustive optimality guarantee. Positive-latency choices require a valid clock/reset context. `choice` is reserved for user-authored equivalent alternatives. Project profiles may set the documented `intent-exploration` count limits; there is no source-level timeout.

See implementation intent, frequency, and II for every accepted metric/objective, synthesis clock control, project-profile configuration, and the exact ready/valid II>1 boundary.

A.7 Verification

```
contract arithmetic @ clk {
    require legal_inputs { (a < 8) & (b < 8) }
    ensure exact_output { y == a + b }
    cover maximum { y == 14 }
}
```

Named `assert`/`ensure` are same-cycle safety goals. `require` is an environment-owned precondition scoped to its contract. `cover` is bounded reachability; a missed cover is not an unreachability proof. BMC produces `bounded_pass`, never `proven`. Verification syntax does not legalize invalid hardware or feed optimizer range inference.

Liveness, arbitrary SVA/SMT, temporal sequences, and source-level equivalence controls are outside this syntax. Start from the existing formal examples.

A.8 Commands and completion check

```
zlang source.zhl --check
zlang source.zhl --top Top --systemverilog build/Top.sv
zlang source.zhl --top Top --target sky130-fd-sc-hd --systemverilog build/Top.sv
zlang source.zhl --top Top --verify
zlang sim source.zhl --top Top --clock clk --cycles 100
zlang sim source.zhl --top Top --logic-state --clock clk --cycles 100
zlang sim source.zhl --top Top --strict-uninitialized --clock clk --cycles 100
zlang verify build/verify --mode bmc --depth 20 --work-dir build/verify-work
```

Ordinary project compilation validates the checked-in lock read-only. Run `zlang lock update --project zlang.toml` only after dependency declarations, dependency source contents, exact Git revisions or other resolution inputs change. A project with no dependencies needs no lock-update prerequisite for simulation or RTL generation. Compilation never fetches or rewrites the lock.

In a `--events` JSONL schedule, `"repeat": N` repeats one event lazily for a bounded positive number of steps (at most 1,000,000 expanded events) without a project-specific generator.

Use `--cycles` only for a one-clock top that does not need an explicit reset sequence. Resettable or multi-clock designs should use JSONL so pin levels, inputs and coincident edges are reproducible:

```
{"reset":{"rst":true},"edges":["clk"]}
{"reset":{"rst":false},"set":{"enable":1},"edges":["clk"]}
{"set":{"enable":0},"edges":["clk"],"repeat":4}
```

Run it with `--events events.jsonl`; add `--trace build/run.fst` and repeated `--trace-signal NAME` options for a selective waveform. Add `--compare-with verilator` or `--compare-with iverilog` to replay the same events against generated Direct-SV. Any mismatch, unresolved RTL output, tool failure, or timeout is an error, not a warning.

Ordinary native simulation retains its deterministic two-state zero seed. `--logic-state` enables per-bit `0/1/U/X`: unreset state starts as `U`, an accepted unresolved write becomes `X`, and controlling values such as `U | 1` still resolve exactly. `--initial-reg` overrides selected initial registers and `--trace-signal` selects waveform signals. Prefer `.fst` for native interactive traces; `.vcd` remains supported for interchange. Hidden `__zlang_meta/*_u_mask` traces distinguish `U` from the formats' shared `x` spelling. `--strict-uninitialized` implies this mode and rejects observed `U` or `X`. Simulation executes the selected implementation in both modes. U/X tags are diagnostics, not physical values; legal binary-equivalent optimization may change their propagation. Strict mode checks typed observations and stateful controls, not every syntactic operand in the original expression. Traces and `get_logic()` may retain unresolved tags for inspection. Neither mode models `Z`, electrical resolution, or Verilog event regions. CLI FST/VCD output is an event-index visualization: physical clock/reset levels match each compiled domain, but the JSONL schedule does not define analog time.